

《机器学习基础》



序列模型、注意力、Transformer

https://github.com/styluck/mech_learn

内容

▶ 序列生成模型

- ▶ 序列概率模型
- ▶ 束搜索
- ▶ 序列到序列模型
- ▶ 评价方法

▶ 注意力机制

- ▶ 注意力模型
- ▶ 自注意力模型
- ▶ 多头注意力模型
- ▶ 多头自注意力模型

▶ Transformer模型



序列建模的基本问题

如何给一个句子赋概率

▶以自然语言为例，

面包上涂黄油

面包上涂袜子

▶规则是概率：自然语言理解 → 一个句子中单词排序的可能性/合理性。

▶序列数据的特点：长度可变、顺序重要、输出空间巨大。

序列概率模型

- ▶ 给定一个序列样本，其概率为

$$p(x_{1:T}) = p(x_1, x_2, \dots, x_T)$$

- ▶ 序列概率模型有两个基本问题：

- ▶ 学习问题：给定一组序列数据，估计这些数据背后的概率分布；
- ▶ 生成问题：从已知的序列分布中生成新的序列样本。

序列概率模型

- ▶ 不能直接建模整个序列，所以要分解

$$\begin{aligned} p(x_{1:T}) &= p(x_1)p(x_2|x_1)p(x_3|x_{1:2}) \cdots p(x_t|x_{1:(t-1)}) \\ &= \prod_t p(x_t|x_{1:t-1}) \end{aligned}$$

- ▶ 因此，序列数据的概率密度估计问题可以转换为单变量的条件概率估计问题，即给定 $x_{1:t-1}$ 时 x_t 的条件概率 $p(x_t|x_{1:t-1})$ 。
- ▶ 学习问题：最大似然估计。
- ▶ 生成问题：从左到右逐步生成。

序列概率模型

►学习：给定N个序列数据 $\{x_{1:T_n}^{(n)}\}_{n=1}^N$ ，序列概率模型需要学习一个模型 $p_\theta(x|x_{1:(t-1)})$ 来最大化整个数据集的对数似然函数。

$$\max_{\theta} \sum_{n=1}^N \sum_{t=1}^{T_n} \log p_\theta \left(x_t^{(n)} \middle| x_{1:(t-1)}^{(n)} \right)$$

自回归生成模型

▶ 自回归模型

$$p(x) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_1, x_2) \cdot \dots p(x_T|x_1, \dots x_{T-1})$$



$$p(x_t|x_1, \dots x_{t-1}) = p(x_t|f(x_1, \dots x_{t-1}))$$

关于以前看到的数据的函数

- ▶ 从概率链式法则看，正序和反序都可以分解；但在语言生成中，正序更符合自然的生成过程和任务设定。

序列生成

- ▶ **生成**：一旦通过最大似然估计训练了模型 $p_{\theta}(x|x_{1:(t-1)})$ ，就可以通过时间顺序来生成一个完整的序列样本。
- ▶ 令 $\hat{x}_t$ 为在第 t 时根据分布 $p_{\theta}(x|\hat{x}_{1:(t-1)})$ 生成的词，

$$\hat{x}_t \sim p_{\theta}(x|\hat{x}_{1:(t-1)})$$

- ▶ 其中 $\hat{x}_{1:(t-1)} = \hat{x}_1, \dots, \hat{x}_{t-1}$ 为前面 $t - 1$ 步中生成的前缀序列。
- ▶ 每一步都需要将前面的输出作为当前步的输入，是一种**自回归** (autoregressive) 的方式。这一类模型也称为**自回归生成模型** (Autoregressive Generative Model)

序列生成

▶ 自回归生成模型 (Autoregressive Generative Model)

▶ 一旦通过最大似然估计训练了模型 $p_{\theta}(x|x_{1:(t-1)})$ ，就可以通过时间顺序来生成一个完整的序列样本。

▶ 令 $\hat{x}_t$ 为在第 t 时根据分布 $p_{\theta}(x|\hat{x}_{1:(t-1)})$ 生成的词，

$$\hat{x}_t \sim p_{\theta}(x|\hat{x}_{1:(t-1)}) \quad \hat{x}_{1:(t-1)} = f(x_1, \dots, x_{t-1})$$

▶ 其中 $\hat{x}_{1:(t-1)} = \hat{x}_1, \dots, \hat{x}_{t-1}$ 为前面 $t - 1$ 步中生成的前缀序列。

▶ 基于前面 $t - 1$ 步的信息，可以生成第 t 步的序列。

▶ 机器翻译、摘要、问答等任务不是凭空生成，而是给定输入后生成输出。因此我们需要条件序列生成模型。



序列到序列模型

序列到序列模型

- ▶ 输出不是孤立生成的，而是必须依赖输入：
 - ▶ 机器翻译、 文本摘要、 问答系统、 对话系统、 语音识别等。
- ▶ 建模变成 $p(y_{1:T} \mid x_{1:S})$ ，而不再是 $p(y_{1:T})$
 - ▶ 条件生成的两个对象：输入序列 $x_{1:S}$ ，输出序列 $y_{1:T}$ 。
 - ▶ 按照输出序列的时间顺序分解：

$$p(y_{1:T} \mid x_{1:S}) = \prod_{t=1}^T p(y_t \mid y_{1:t-1}, x_{1:S}).$$

Seq2seq 模型

- ▶ Seq2seq 模型的目标是估计条件概率

$$p_{\theta}(y_{1:T}|x_{1:S}) = \prod_{t=1}^T p_{\theta}(y_t|y_{1:(t-1)}, x_{1:S}),$$

- ▶ 其中 $y_t \in \mathcal{V}$ 为词表 $\mathcal{V}$ 中的某个词。
- ▶ 给定一组训练数据 $\{(x_{S_n}, y_{T_n})\}_{n=1}^N$ ，我们可以使用最大似然估计来训练模型参数。

$$\max_{\theta} \sum_{n=1}^N \log p_{\theta}(y_{1:T_n}|x_{1:S_n}).$$

- ▶ 一旦训练完成，模型就可以根据一个输入序列 x 来生成最可能的目标序列，

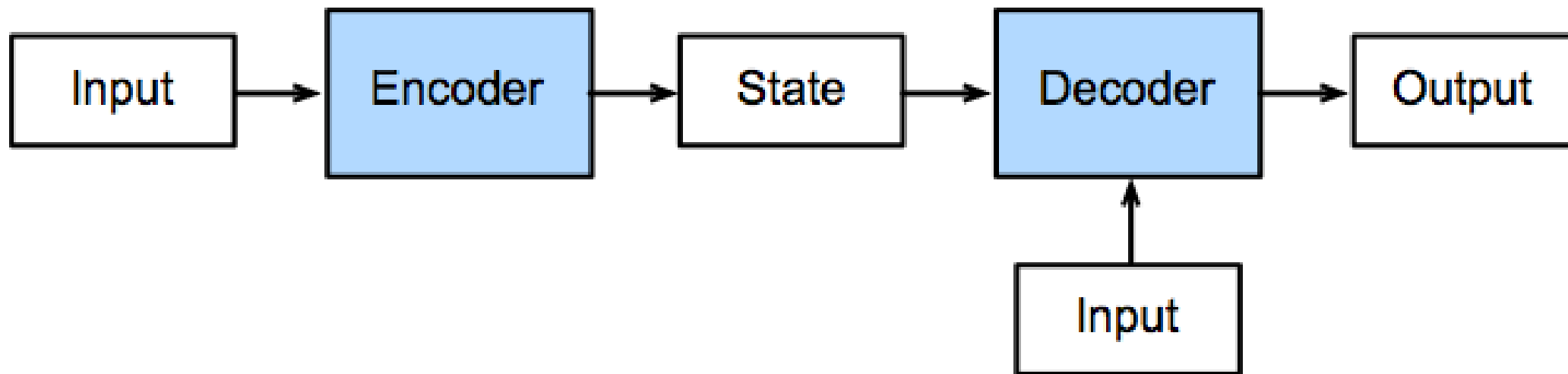
$$\hat{y} = \arg \max_y p_{\theta}(y|x),$$

- ▶ 如何建模？如何生成？

编码器 - 解码器架构

▶ 模型分为两部分

- ▶ 编码器 (Encoder) 加工输入
- ▶ 解码器 (Decoder) 生成输出



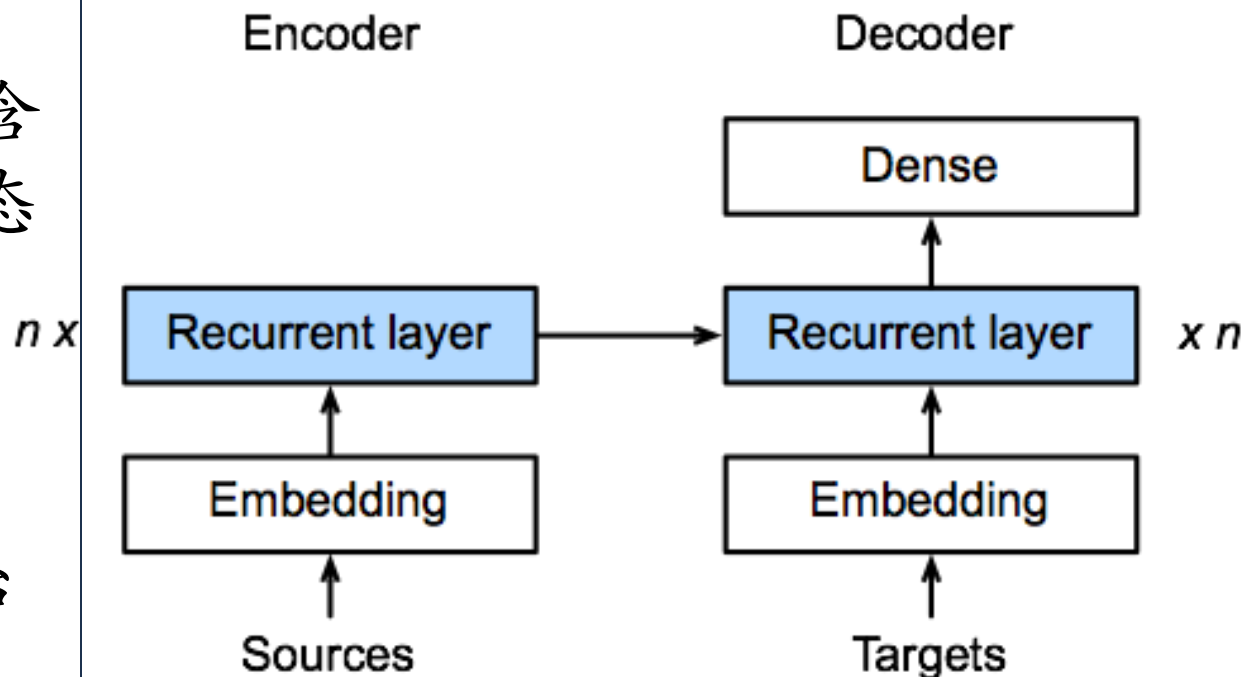
Seq2seq 模型

► 编码器是没有输出层的标准 RNN 模型

► 编码器在上一时间步骤中的隐含状态用作解码器的初始隐藏状态

$$u = \text{Encoder}(x_{1:s}),$$

► Decoder 再以 u 为初始状态开始生成输出。



Seq2seq 模型

► 编码器：

- 首先使用一个循环神经网络 f_{enc} 来编码输入序列 $x_{1:S}$ 得到一个固定维数的向量 u ， u 一般为编码循环神经网络最后时刻的隐状态。

$$h_t^{\text{enc}} = f_{\text{enc}}(h_{t-1}^{\text{enc}}, e_{x_t}, \theta_{\text{enc}}), \quad \forall t \in [1:S],$$
$$u = h_S^{\text{enc}},$$

- 其中 $f_{\text{enc}}(\cdot)$ 为编码循环神经网络，可以为LSTM或GRU，其参数为 θ_{enc} ， e_x 为词 x 的词向量。

Seq2seq 模型

► 解码器：

- 在生成目标序列时，使用另外一个循环神经网络 f_{dec} 来进行解码。
在解码过程的第 t 步时，已生成前缀序列为 $y_{1:(t-1)}$ 。

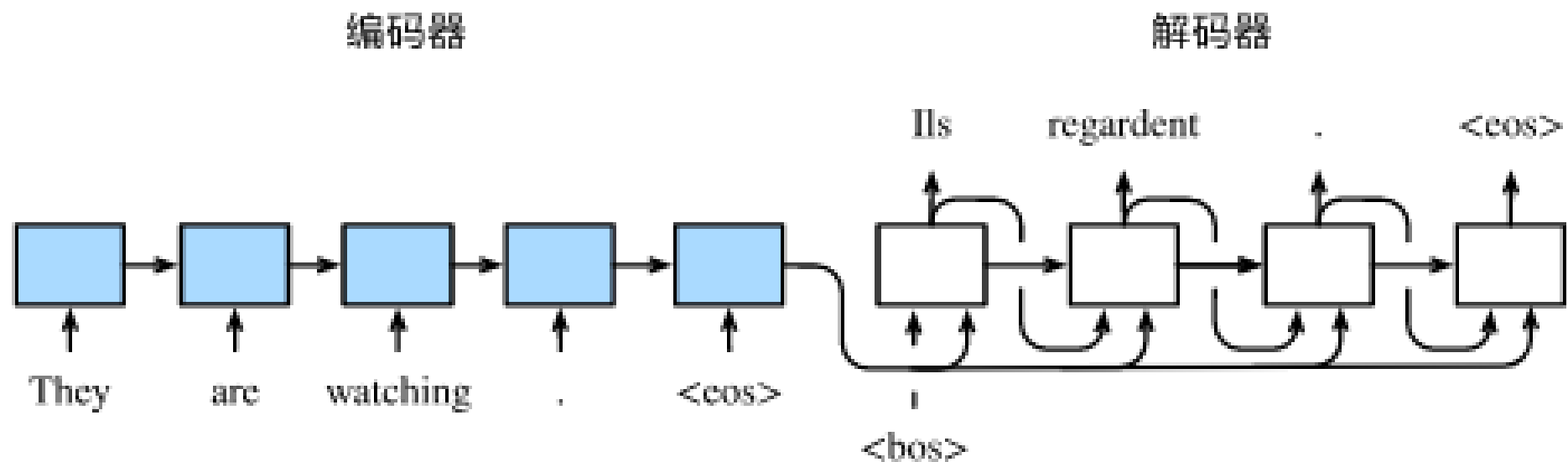
- 令 h_t^{dec} 表示在网络 f_{dec} 的隐状态， $o_t \in (0,1)^{|V|}$ 为词表中所有词的后验概率，则

$$\begin{aligned}h_0^{\text{dec}} &= u, \\h_t^{\text{dec}} &= f_{\text{dec}}(h_{t-1}^{\text{dec}}, e_{y_{t-1}}, \theta_{\text{dec}}), \\o_t &= g(h_t^{\text{dec}}, \theta_o),\end{aligned}$$

- 其中 $f_{\text{dec}}(\cdot)$ 为解码循环神经网络， $g(\cdot)$ 为最后一层为Softmax函数的前馈神经网络， θ_{dec} 和 θ_o 为网络参数， e_y 为 y 的词向量， y_0 为一个特殊符号，比如<EOS>。

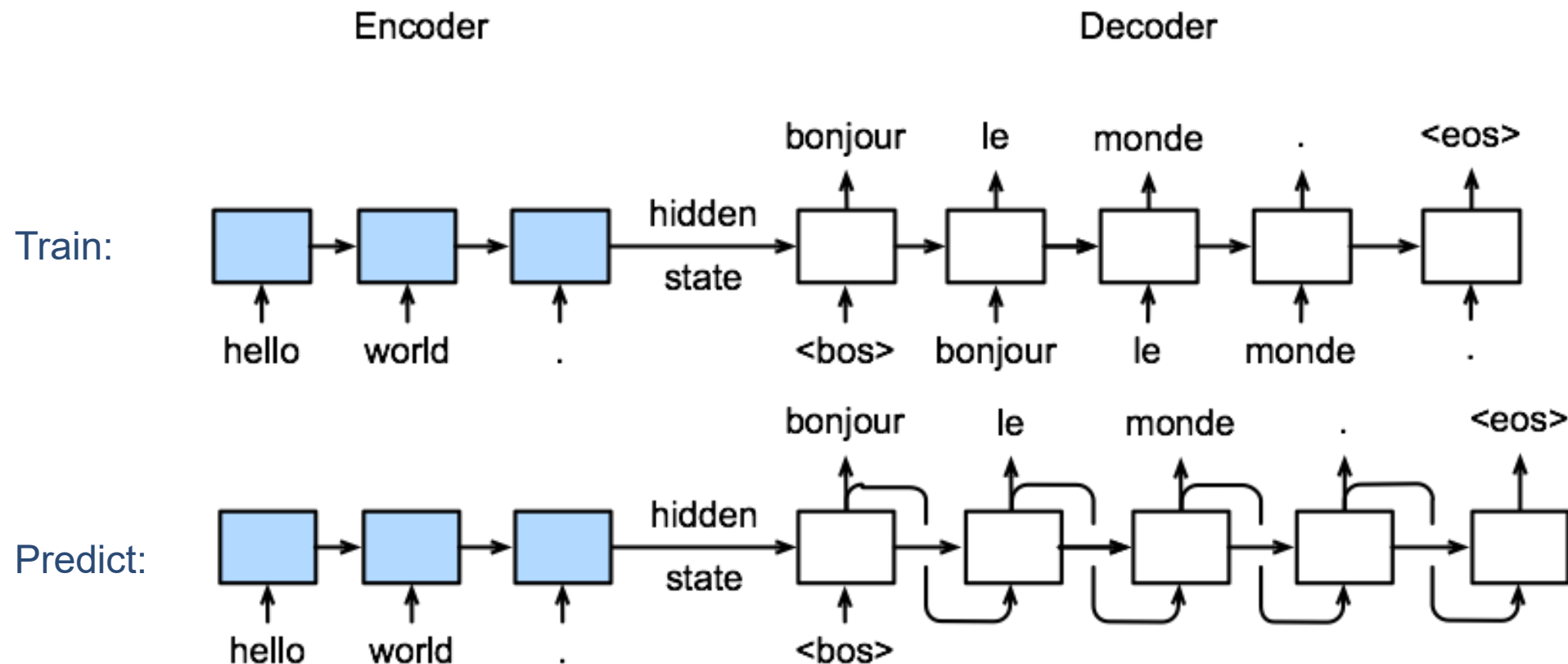
Seq2seq 模型

- ▶ 编码器是读取输入序列的 RNN
- ▶ 解码器使用另一个 RNN 来生成输出



机器翻译训练

► 在训练期间，解码器 (Decoder) 用目标语言句子作为输入



生成最可能序列

► 解码问题：如何找到最可能的输出序列？

$$\text{目标：} \hat{y} = \arg \max_y p_{\theta}(y|x),$$

- 当使用自回归模型生成一个最可能的序列时，生成过程是一种从左到右的贪婪式搜索过程。在每一步都生成最可能的词。
- 这种贪婪式的搜索方式是次优的，生成的序列并不保证是全局最优的：

$$\prod_{t=1}^T \max_{x_t \in \mathcal{V}^T} p_{\theta}(x_t | \hat{x}_{1:(t-1)}) \leq \max_{x_{1:T} \in \mathcal{V}^T} \prod_{t=1}^T p_{\theta}(x | x_{1:(t-1)}).$$

贪婪搜索(Greedy search)

- ▶ 在预测期间在 seq2seq 模型中使用贪婪搜索
- ▶ 它可能不是最理想的

贪婪搜索:

$$0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$$

时间步	1	2	3	4
A	0.5	0.1	0.2	0.0
B	0.2	0.4	0.2	0.2
C	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

更好的选择:

$$0.5 \times 0.3 \times 0.6 \times 0.6 = 0.054$$

时间步	1	2	3	4
A	0.5	0.1	0.1	0.1
B	0.2	0.4	0.6	0.2
C	0.2	0.3	0.2	0.1
<eos>	0.1	0.2	0.1	0.6

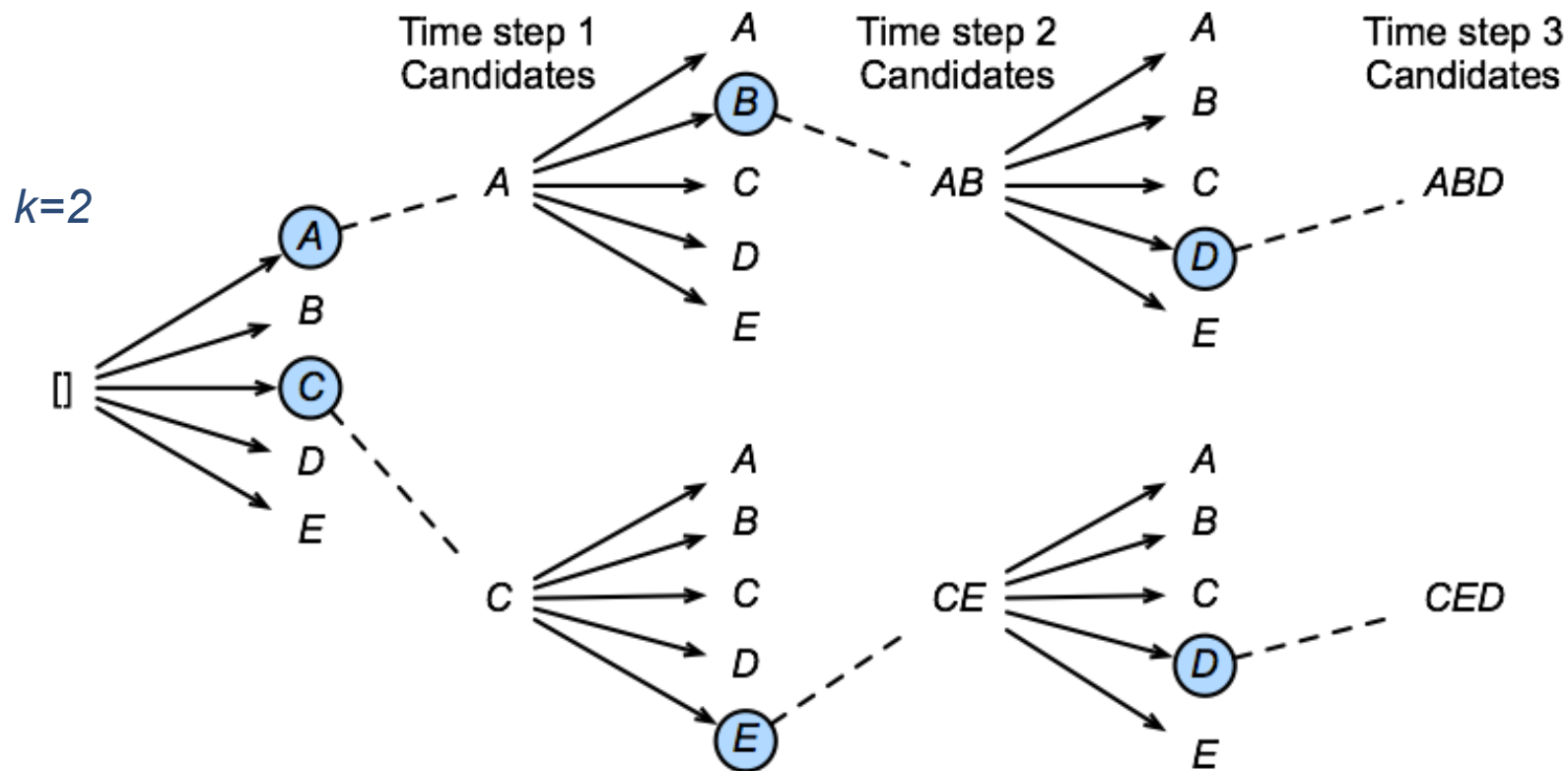
穷举搜索 (exhaustive search)

- ▶ 对于每个可能的序列，计算其概率并选择最佳序列
- ▶ 如果输出词汇量大小为 n ，并且最大序列长度为 T ，那么我们需要检查 n^T 序列
- ▶ 这在计算上是不可行的

$$n = 10000, T = 10: n^T = 10^{40}$$

束搜索

- ▶ 每次都保留最好的 k (束搜索) 候选
- ▶ 通过向候选束添加新项目, 来搜索 $k|V|$ 个候选输出, 然后保留前 k 个最可能的序列.



束搜索

- ▶ 时间复杂度为 $O(knT)$

$$k = 5, n = 10000, T = 10:$$

$$knT = 5 \times 10^5$$

$$n = |\mathcal{V}|$$

- ▶ 每个候选的最终得分是

$$\frac{1}{L^\alpha} \log \mathbb{P}(y_1, \dots, y_L) = \frac{1}{L^\alpha} \sum_{t'=1}^L \log \mathbb{P}(y_{t'} \mid y_1, \dots, y_{t'-1}, \mathbf{c})$$

- ▶ 常用值 $\alpha = 0.75$



注意力机制

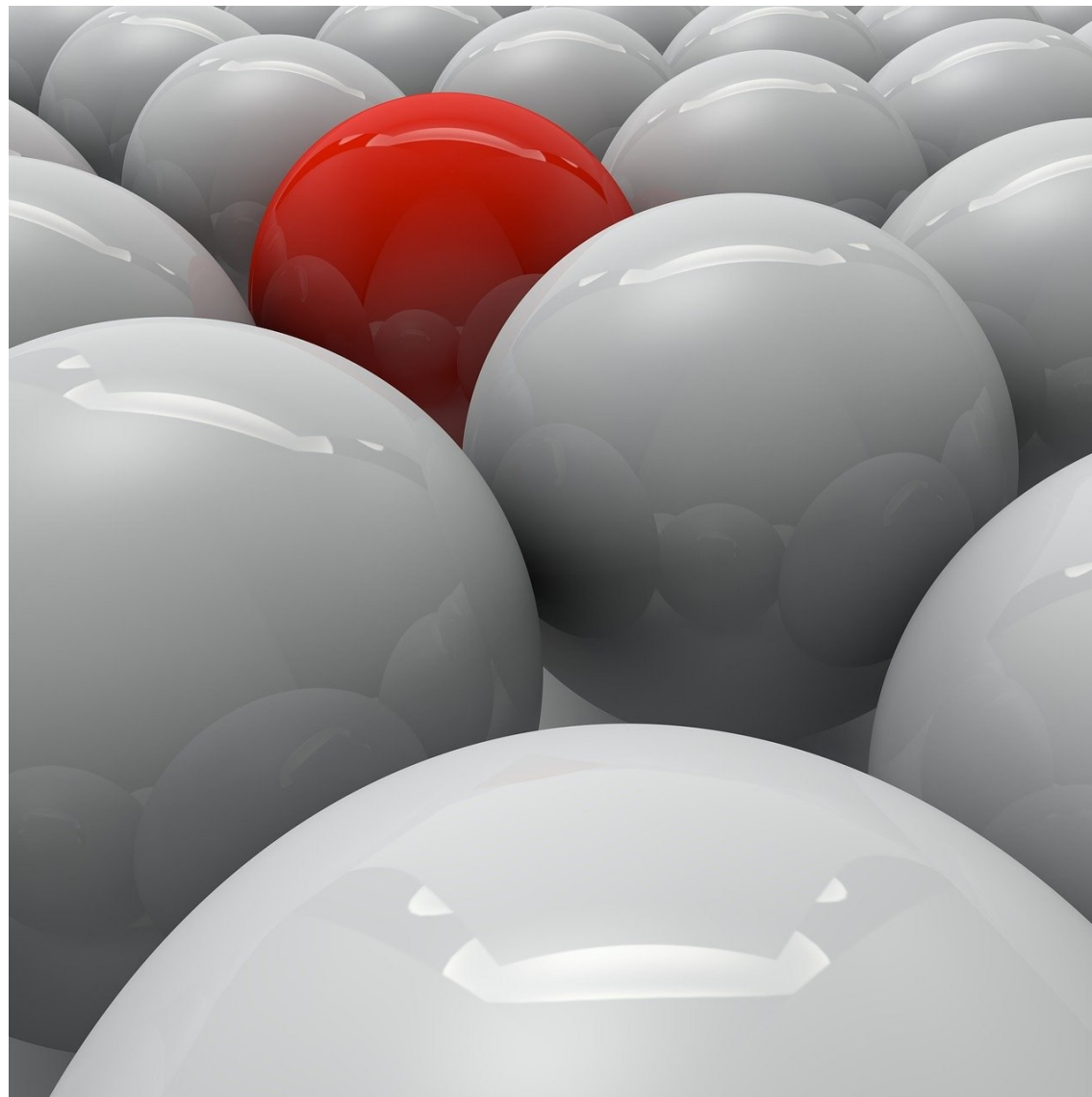
RNN-Seq2Seq 的瓶颈

- ▶ 编码器把整个输入序列压缩成一个固定长度向量。
- ▶ RNN-Seq2Seq 的流程:
- ▶ 整个输入序列被压缩成一个固定长度向量。句子越长，信息损失越严重。
- ▶ 解码时，不同输出词其实应该关注输入句子的不同位置。
- ▶ 如何提高网络能力？

大脑中的注意力

- ▶ 人脑每个时刻接收的外界输入信息非常多，包括来源于视觉、听觉、触觉的各种各样的信息。
- ▶ 但就视觉来说，眼睛每秒钟都会发送千万比特的信息给视觉神经系统。
- ▶ 人脑通过**注意力**来解决**信息超载**问题。

注意力示例



例子：阅读理解 SQuAD

The first recorded travels by Europeans to China and back date from this time. The most famous traveler of the period was the Venetian Marco Polo, whose account of his trip to "Cambaluc," the capital of the Great Khan, and of life there astounded the people of Europe. The account of his travels, *Il milione* (or, *The Million*, known in English as the *Travels of Marco Polo*), appeared about the year 1299. Some argue over the accuracy of Marco Polo's accounts due to the lack of mentioning the Great Wall of China, tea houses, which would have been a prominent sight since Europeans had yet to adopt a tea culture, as well the practice of foot binding by the women in capital of the Great Khan. Some suggest that Marco Polo acquired much of his knowledge through contact with Persian traders since many of the places he named were in Persian.

How did some suspect that Polo learned about China instead of by actually visiting it?

例子：阅读理解 SQuAD

The first recorded travels by Europeans to China and back date from this time. The most famous traveler of the period was the Venetian Marco Polo, whose account of his trip to "Cambaluc," the capital of the Great Khan, and of life there astounded the people of Europe. The account of his travels, Il milione (or, The Million, known in English as the Travels of Marco Polo), appeared about the year 1299. Some argue over the accuracy of Marco Polo's accounts due to the lack of mentioning the Great Wall of China, tea houses, which would have been a prominent sight since Europeans had yet to adopt a tea culture, as well the practice of foot binding by the women in capital of the Great Khan. Some suggest that Marco Polo acquired much of his knowledge through contact with Persian traders since many of the places he named were in Persian.

How did some suspect that Polo learned about China instead of by actually visiting it?

Answer: through contact with Persian traders

注意力示例



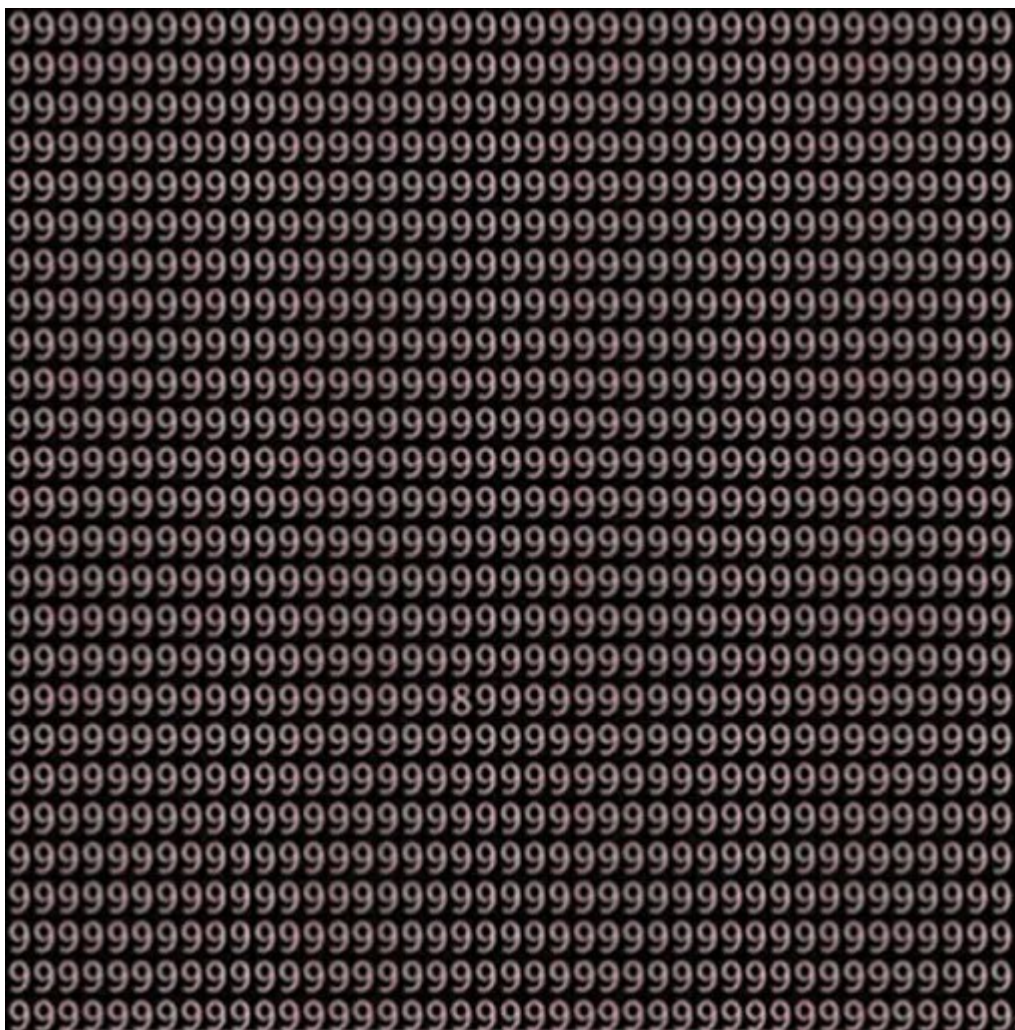
所有黑色牌的数字之和为多少？

注意力示例

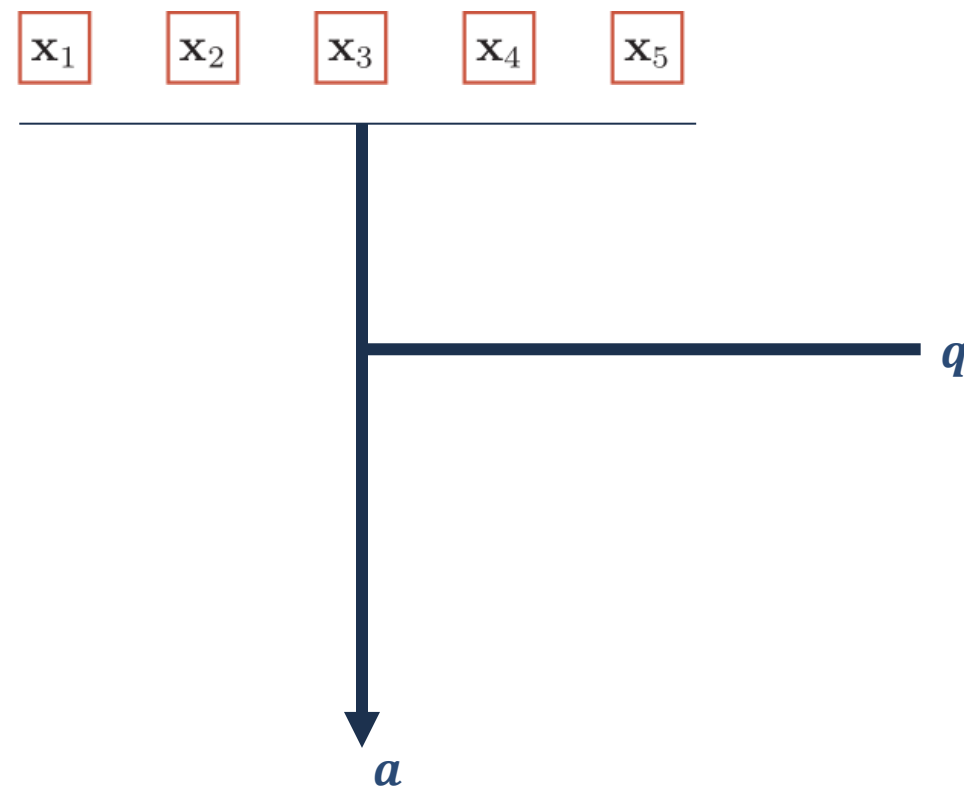
- ▶ 当一个人在吵闹的鸡尾酒会上和朋友聊天时，尽管周围噪音干扰很多，他还是可以听到朋友的谈话内容，而忽略其他人的声音。

鸡尾酒会效应

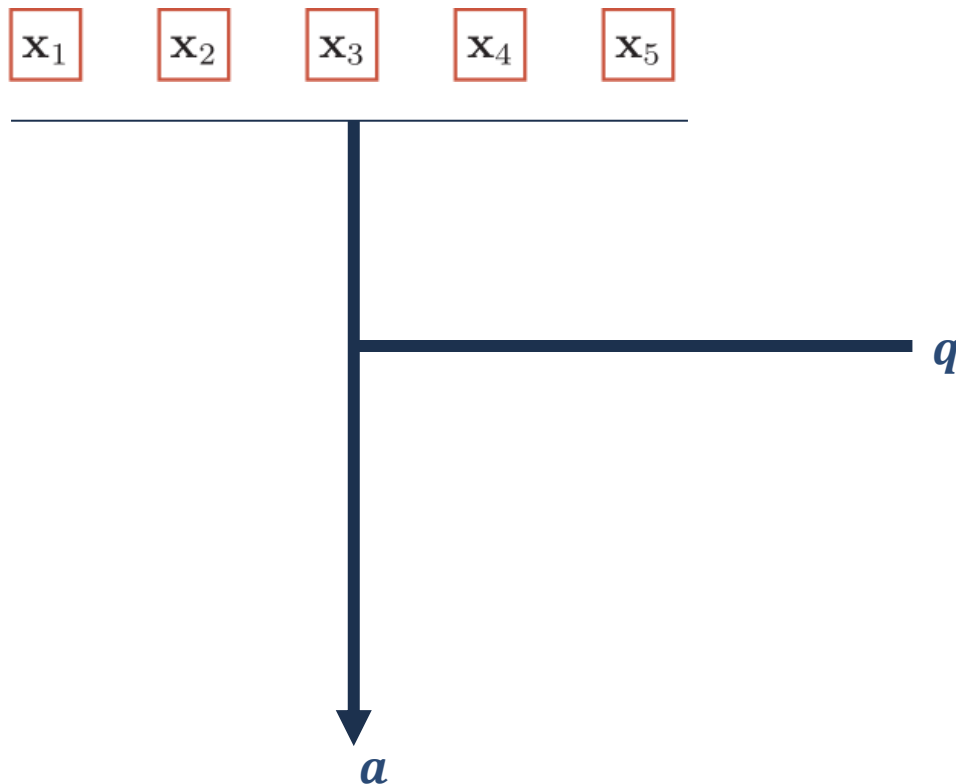
问题



Try to find 8 less than 8 seconds



问题



► 输入：

- 用 $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N] \in \mathbb{R}^{D \times N}$ 表示 N 组输入信息，其中 D 维向量 $\mathbf{x}_n \in \mathbb{R}^D, n \in [1, N]$ 表示一组输入信息。
- 从 N 个输入向量中选择出和某个特定任务相关的信息，我们需要引入一个和任务相关的表示，称为查询向量 q

► 输出：

- 在给定 q 和 $\mathbf{X}$ 下，选择第 n 个输入向量的概率 α_n

如何实现？

- ▶ 全连接：不适合变长序列。
- ▶ CNN/RNN 处理序列的局限：局部依赖或长距离依赖困难。
- ▶ RNN-Seq2Seq：输入序列被压缩，信息损失越严重。
- ▶ 注意力：不再只依赖最后一个隐藏状态 h_S^{enc} ，而是保留 Encoder 每个位置的隐藏状态，Decoder 在每一步生成时，动态决定应该关注哪些输入位置。

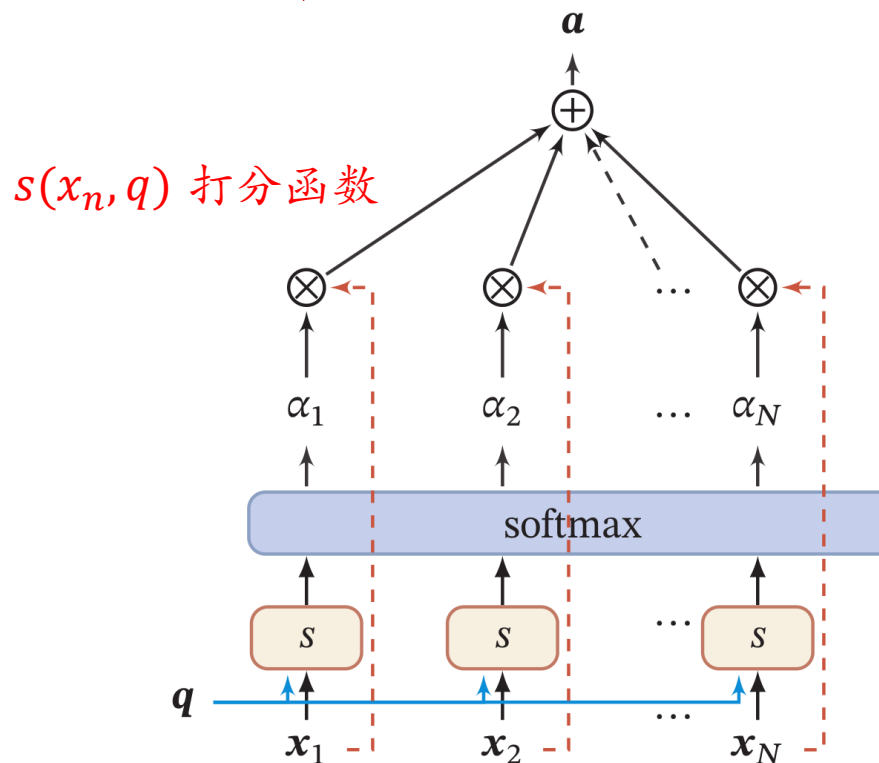
注意力模型

- ▶ 硬性注意力机制 (hard attention mechanism)
- ▶ 软性注意力机制 (soft attention mechanism)
- ▶ 注意力机制可以分为两步

- ▶ 计算注意力分布 α ,
$$\begin{aligned}\alpha_n &= p(z = n | \mathbf{X}, \mathbf{q}) \\ &= \text{softmax}(s(\mathbf{x}_n, \mathbf{q})) \\ &= \frac{\exp(s(\mathbf{x}_n, \mathbf{q}))}{\sum_{j=1}^N \exp(s(\mathbf{x}_j, \mathbf{q}))}\end{aligned}$$

- ▶ 根据 α 来计算输入信息的加权平均。

$$\begin{aligned}\text{att}(\mathbf{X}, \mathbf{q}) &= \sum_{n=1}^N \alpha_n \mathbf{x}_n, \\ &= \mathbb{E}_{z \sim p(z | \mathbf{X}, \mathbf{q})}[\mathbf{x}_z]\end{aligned}$$



注意力打分函数 $s(x_n, q)$

加性模型

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{v}^\top \tanh(\mathbf{W}\mathbf{x} + \mathbf{U}\mathbf{q}),$$

点积模型

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{x}^\top \mathbf{q},$$

缩放点积模型

$$s(\mathbf{x}, \mathbf{q}) = \frac{\mathbf{x}^\top \mathbf{q}}{\sqrt{D}},$$

双线性模型

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{x}^\top \mathbf{W} \mathbf{q},$$

缩放点积模型向量化版本

$$s(\mathbf{K}, \mathbf{Q}) = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}$$

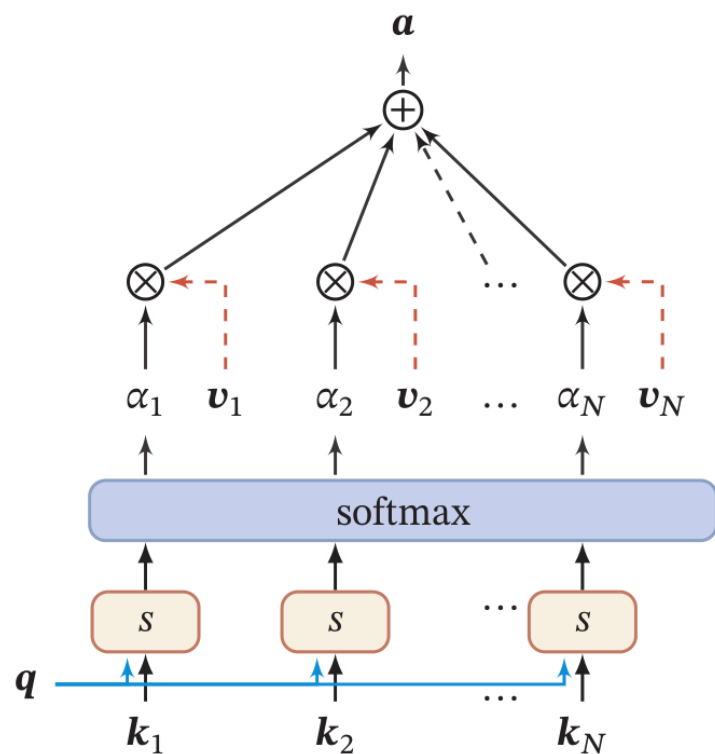
m 个询问 $\mathbf{Q} \in \mathbb{R}^{m \times d}$ 和 n 个键 $\mathbf{K} \in \mathbb{R}^{n \times d}$

多层感知注意力

$$s(\mathbf{k}, \mathbf{v}, \mathbf{q}) = \mathbf{v}^\top \tanh(\mathbf{W}_k \mathbf{k} + \mathbf{W}_q \mathbf{q})$$

注意力机制的变体

► 键值对注意力 (key-value pair attention)

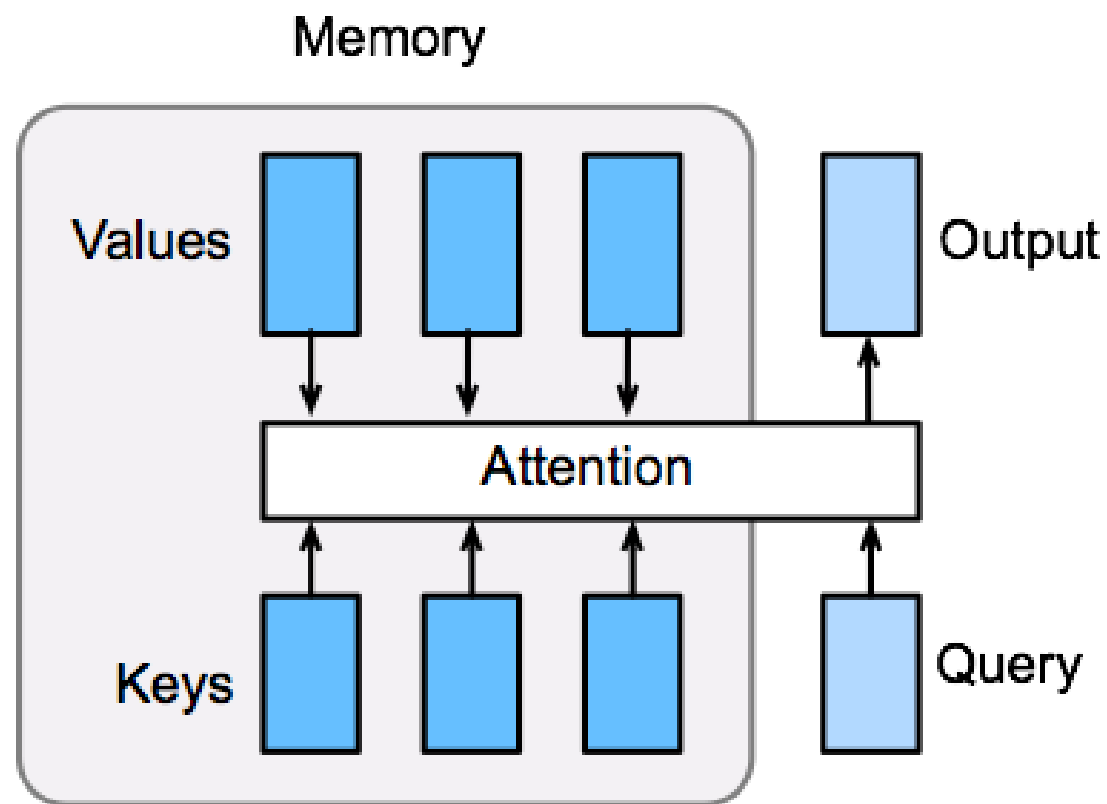


用 $(K, V) = [(k_1, v_1), \dots, (k_N, v_N)]$
表示 N 个输入信息

$$\begin{aligned} \text{att}((K, V), q) &= \sum_{n=1}^N \alpha_n v_n, \\ &= \sum_{n=1}^N \frac{\exp(s(k_n, q))}{\sum_j \exp(s(k_j, q))} v_n \end{aligned}$$

注意力层

- ▶ **注意力层明确选择相关信息**
 - ▶ 它的存储器（memory）由“键值对”组成
 - ▶ 键和查询越相似，则输出的值越相近



多层感知注意力(键值对注意力)

► 可学习的参数 $\mathbf{W}_k \in \mathbb{R}^{h \times d_k}$, $\mathbf{W}_q \in \mathbb{R}^{h \times d_q}$, $\mathbf{v} \in \mathbb{R}^h$

$$s(\mathbf{k}, \mathbf{v}, \mathbf{q}) = \mathbf{v}^T \tanh(\mathbf{W}_k \mathbf{k} + \mathbf{W}_q \mathbf{q})$$

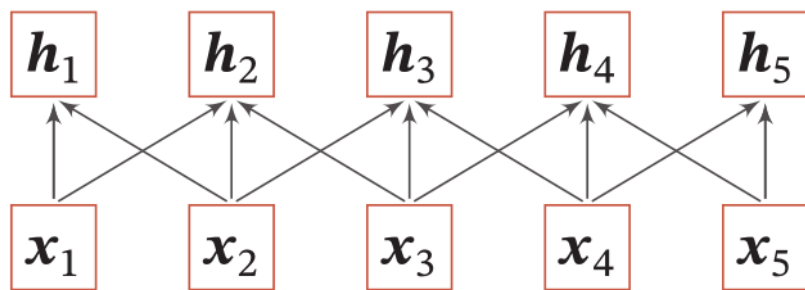
► 相当于连接“键”(key)和“询问”(query), 然后输入隐含大小为 h 和输出大小 1 的单个隐含层感知



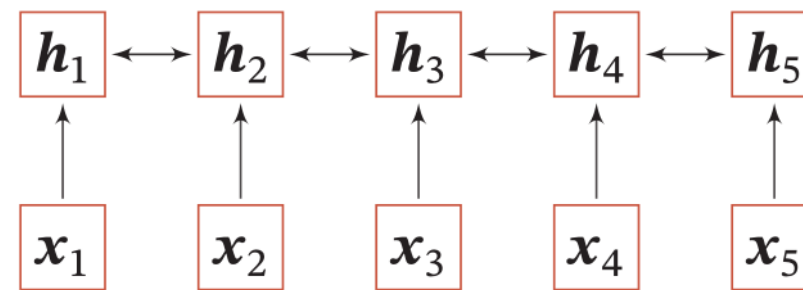
自注意力模型

自注意力模型

- 当使用神经网络来处理一个变长的向量序列时，我们通常可以使用卷积网络或循环网络进行编码来得到一个相同长度的输出向量序列。



(a) 卷积网络



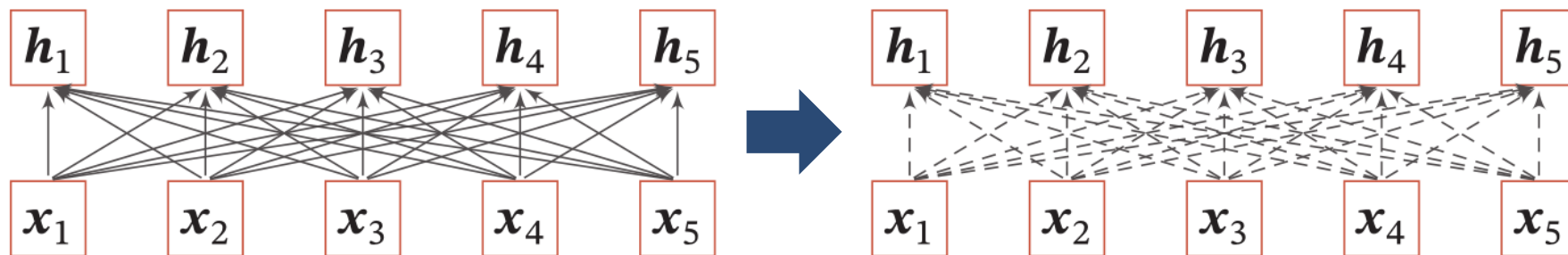
(b) 双向循环网络

只建模了输入信息的局部依赖关系

自注意力模型

► 如何建立非局部 (Non-local) 的依赖关系

► 全连接?



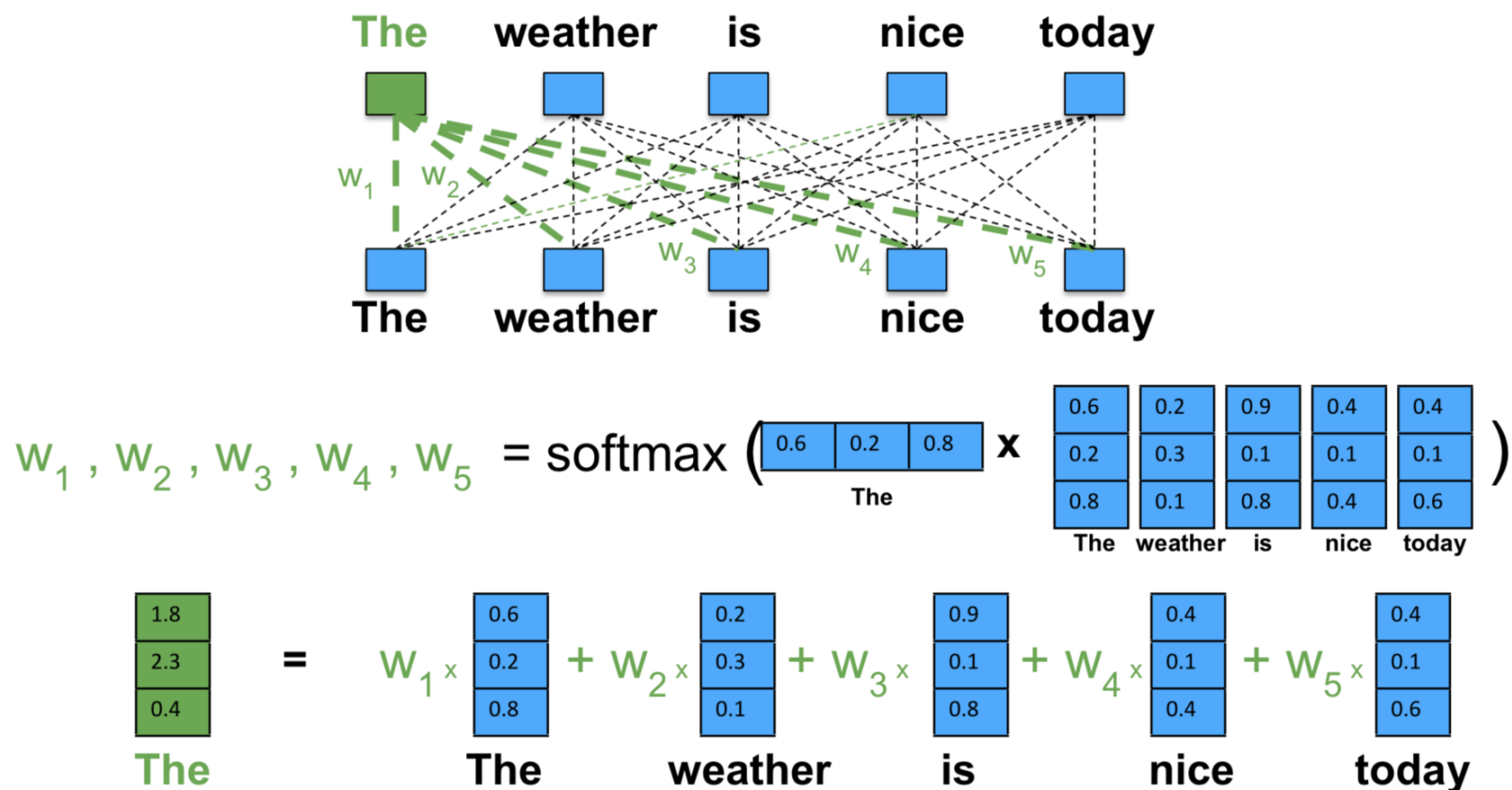
(a) 全连接模型

(b) 自注意力模型

无法处理变长问题

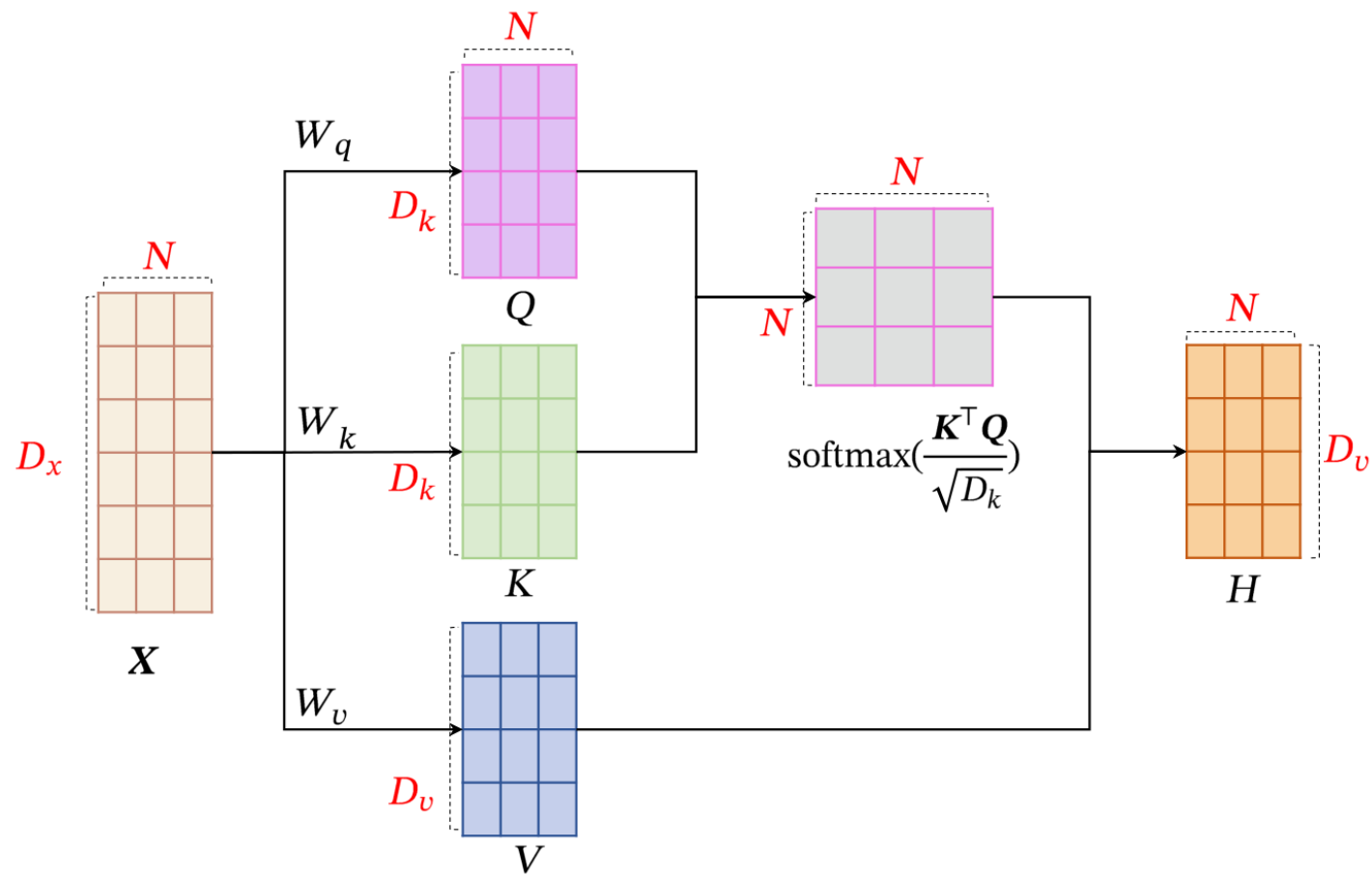
连接权重 α_{ij} 由注意力机制动态生成

自注意力示例



图片来源: http://fuyw.top/NLP_02_QANet/

QKV模式 (Query-Key-Value)



自注意力模型

► 输入序列为 $X = [\mathbf{x}_1, \dots, \mathbf{x}_N] \in \mathbb{R}^{D_x \times N}$

► 首先生成三个向量序列

$$\mathbf{Q} = \mathbf{W}_q \mathbf{X} \in \mathbb{R}^{D_k \times N},$$

$$\mathbf{K} = \mathbf{W}_k \mathbf{X} \in \mathbb{R}^{D_k \times N},$$

$$\mathbf{V} = \mathbf{W}_v \mathbf{X} \in \mathbb{R}^{D_v \times N},$$

► 计算 $\mathbf{h}_n$

$$\mathbf{h}_n = \text{att}((\mathbf{K}, \mathbf{V}), \mathbf{q}_n)$$

► 如果使用缩放点积来作为注意力打分函数，输出向量序列可以简写为

$$\mathbf{H} = \mathbf{V} \text{softmax}\left(\frac{\mathbf{K}^\top \mathbf{Q}}{\sqrt{D_k}}\right),$$

多头 (multi-head) 注意力

- ▶ 单个注意力头只能从一种相似性角度聚合信息。
 - ▶ 但语言中可能同时存在语法关系、指代关系、位置关系、语义关系。
 - ▶ 我们希望模型同时从多个子空间学习不同类型的依赖关系。
- ▶ 多头注意力 (multi-head attention)
 - ▶ 利用多个查询 $\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_M]$ ，来并行地从输入信息中选取多组信息。每个注意力关注输入信息的不同部分。

$$\text{att}((\mathbf{K}, \mathbf{V}), \mathbf{Q}) = \text{att}((\mathbf{K}, \mathbf{V}), \mathbf{q}_1) \oplus \dots \oplus \text{att}((\mathbf{K}, \mathbf{V}), \mathbf{q}_M)$$

多头 (multi-head) 自注意力

▶ 多头自注意力 (multi-head self-attention)

- ▶ 结合了自注意力、多头注意力
- ▶ 加入了位置编码，扩展了模型专注于不同位置的能力

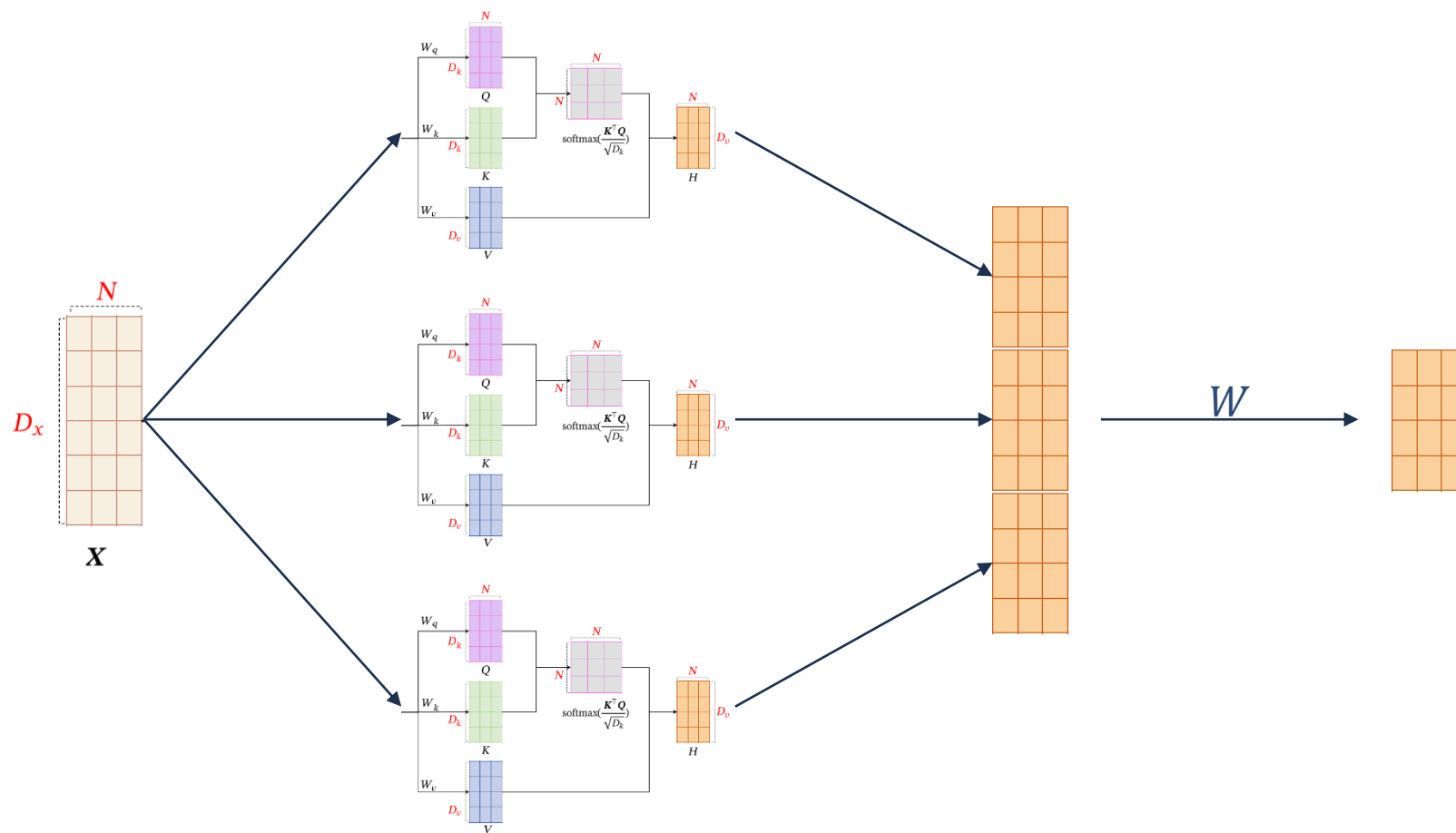
$$\text{MultiHead}(\mathbf{H}) = \mathbf{W}_o[\text{head}_1; \dots; \text{head}_M],$$

$$\text{head}_m = \text{self-att}(\mathbf{Q}_m, \mathbf{K}_m, \mathbf{V}_m),$$

$$\forall m \in \{1, \dots, M\}, \quad \mathbf{Q}_m = \mathbf{W}_q^m \mathbf{H}, \mathbf{K} = \mathbf{W}_k^m \mathbf{H}, \mathbf{V} = \mathbf{W}_v^m \mathbf{H},$$

其中 $\mathbf{W}_o \in \mathbb{R}^{D_h \times M d_v}$ 为输出投影矩阵, $\mathbf{W}_q^m \in \mathbb{R}^{D_k \times D_h}$, $\mathbf{W}_k^m \in \mathbb{R}^{D_k \times D_h}$, $\mathbf{W}_v^m \in \mathbb{R}^{D_v \times D_h}$ 为投影矩阵, $m \in \{1, \dots, M\}$.

多头 (multi-head) 自注意力模型



多头 (multi-head) 自注意力模型

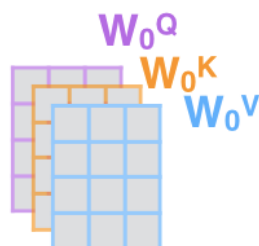
1) This is our input sentence*

Thinking
Machines

2) We embed each word*



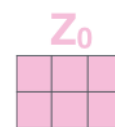
3) Split into 8 heads. We multiply X or R with weight matrices



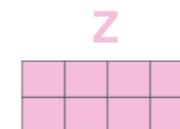
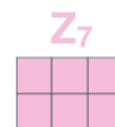
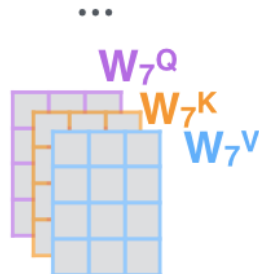
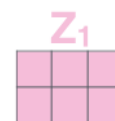
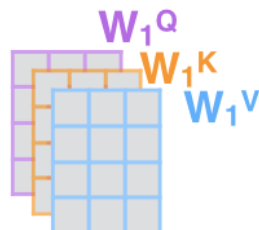
4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



图片来源: <http://jalammar.github.io/illustrated-transformer/>

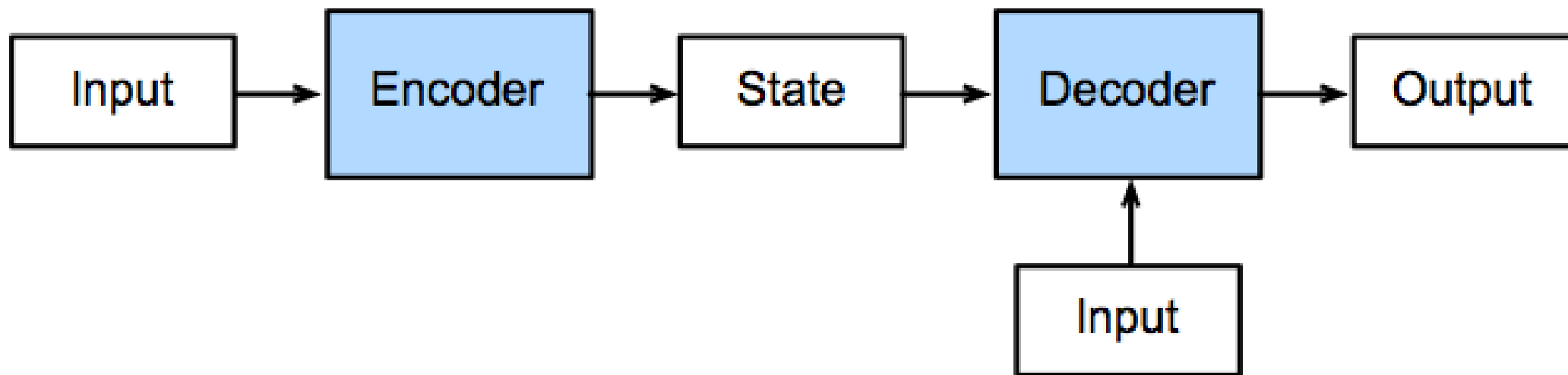


Transformer

编码器 - 解码器架构

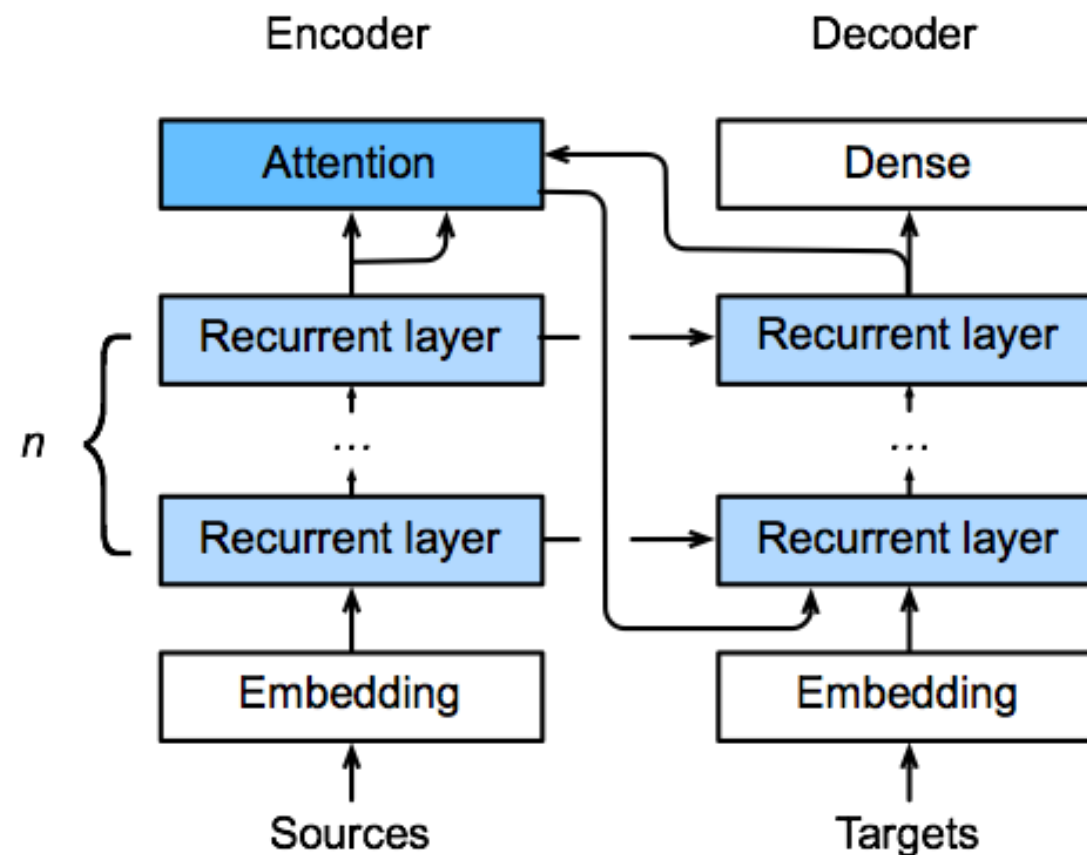
▶ 模型分为两部分

- ▶ 编码器 (Encoder) 加工输入
- ▶ 解码器 (Decoder) 生成输出



编码器 - 解码器上的注意力机制

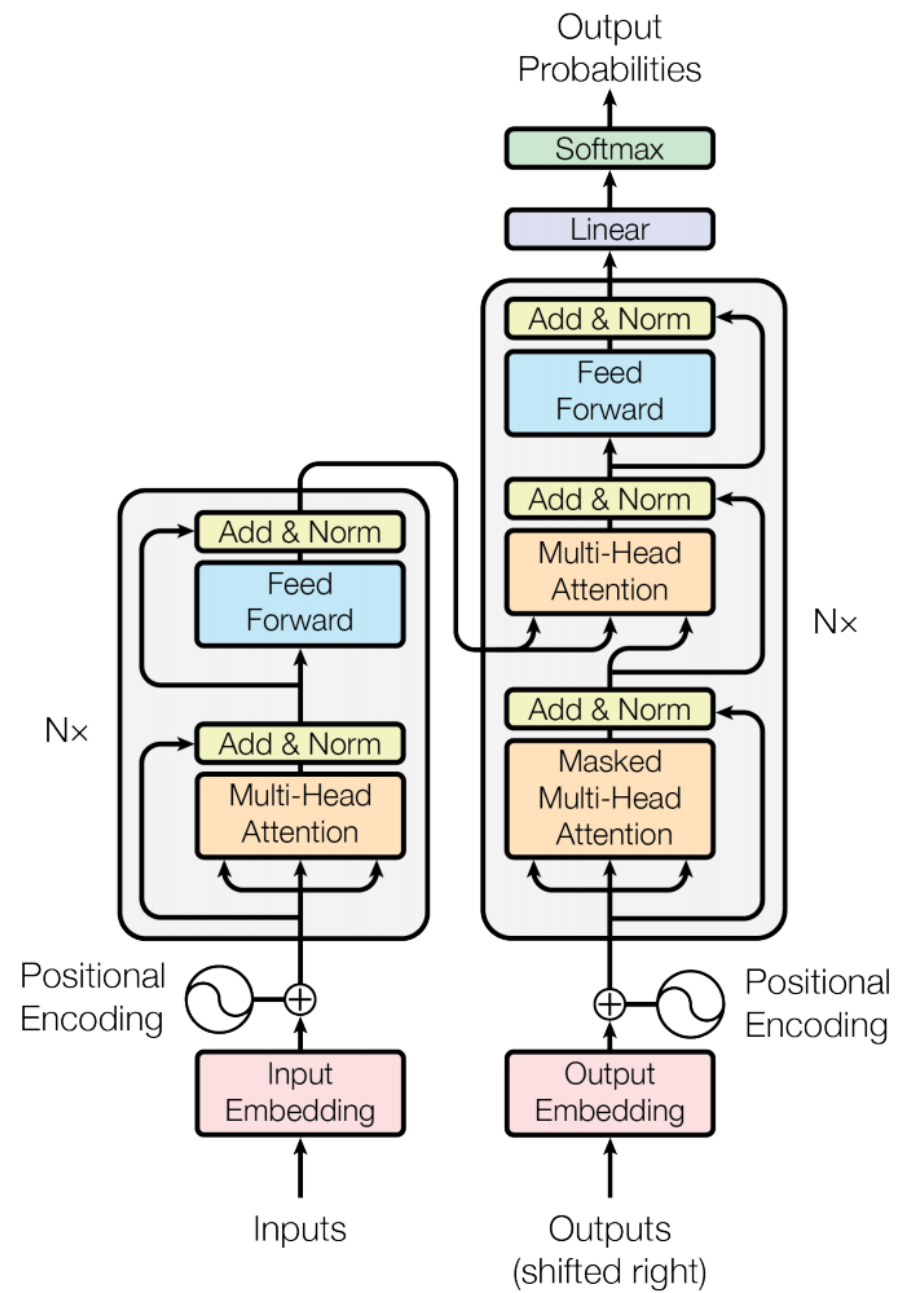
- ▶ 使用编码器中最后一个循环神经网络层的输出。
- ▶ 然后，注意力输出与嵌入输入拼接，以输入解码器中的第一个循环神经网络层。



Attention Is All You Need

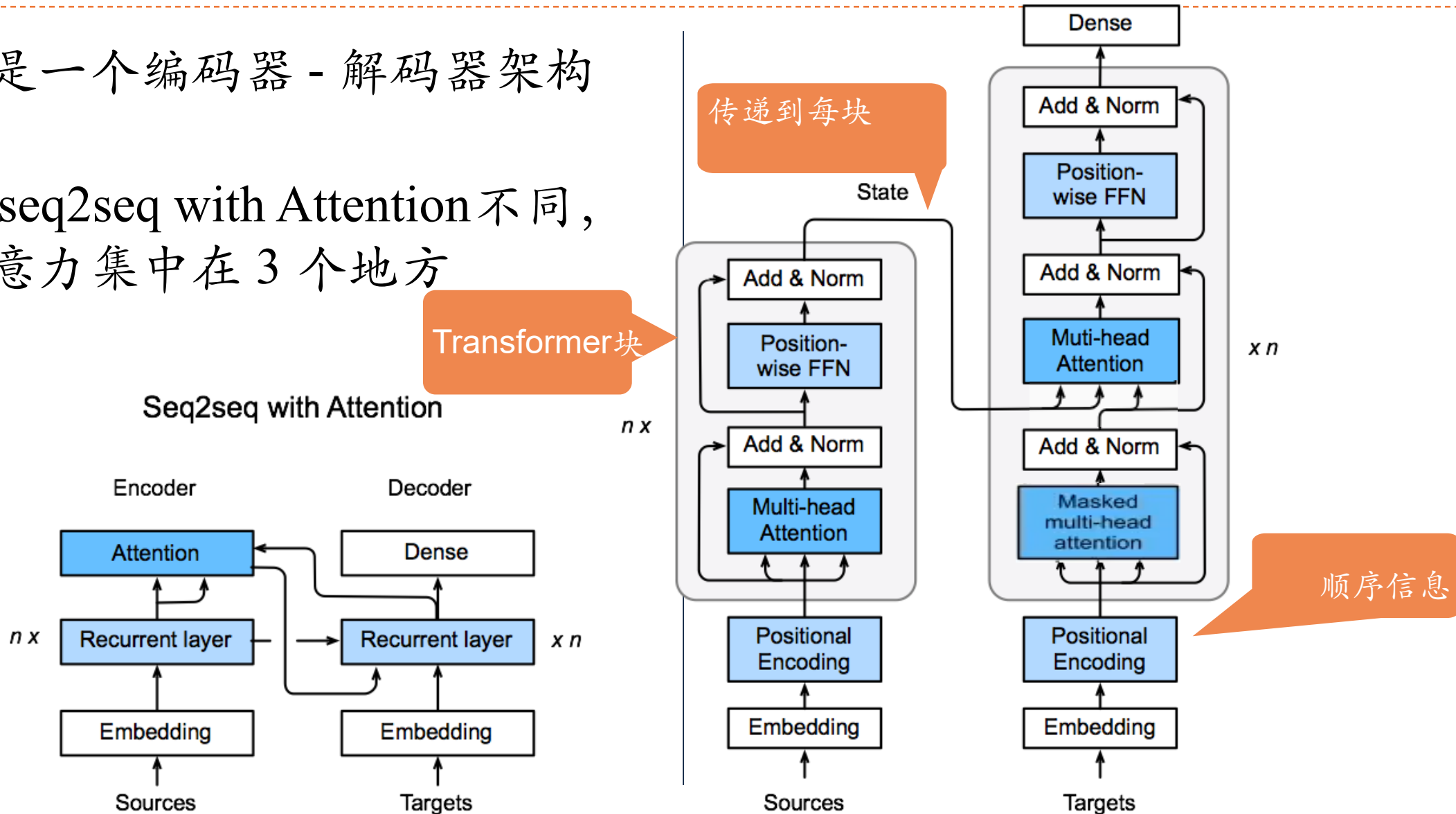
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.



Transformer模型架构

- ▶ 它是一个编码器 - 解码器架构
- ▶ 与 seq2seq with Attention 不同，注意力集中在 3 个地方



位置编码

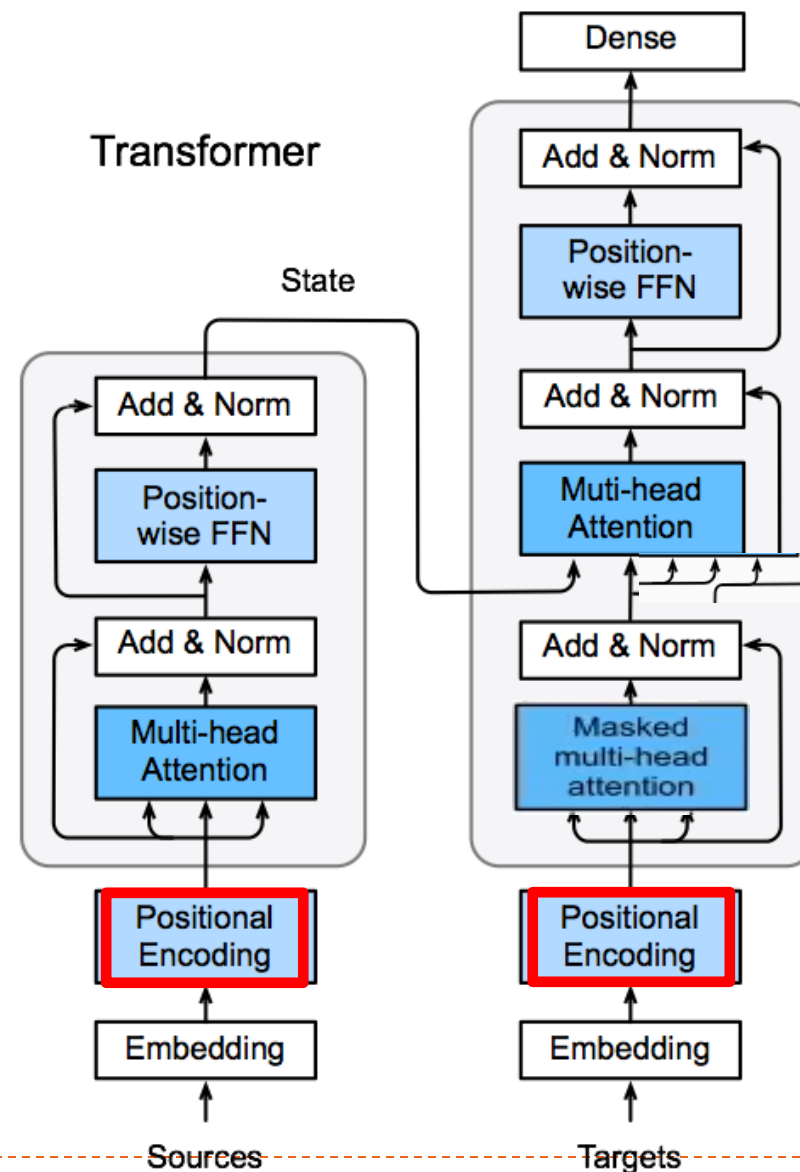
► 假设嵌入 $X \in \mathbb{R}^{l \times d}$ 输出的形状（序列长度，嵌入维度）

► 创建 $P \in \mathbb{R}^{l \times d}$

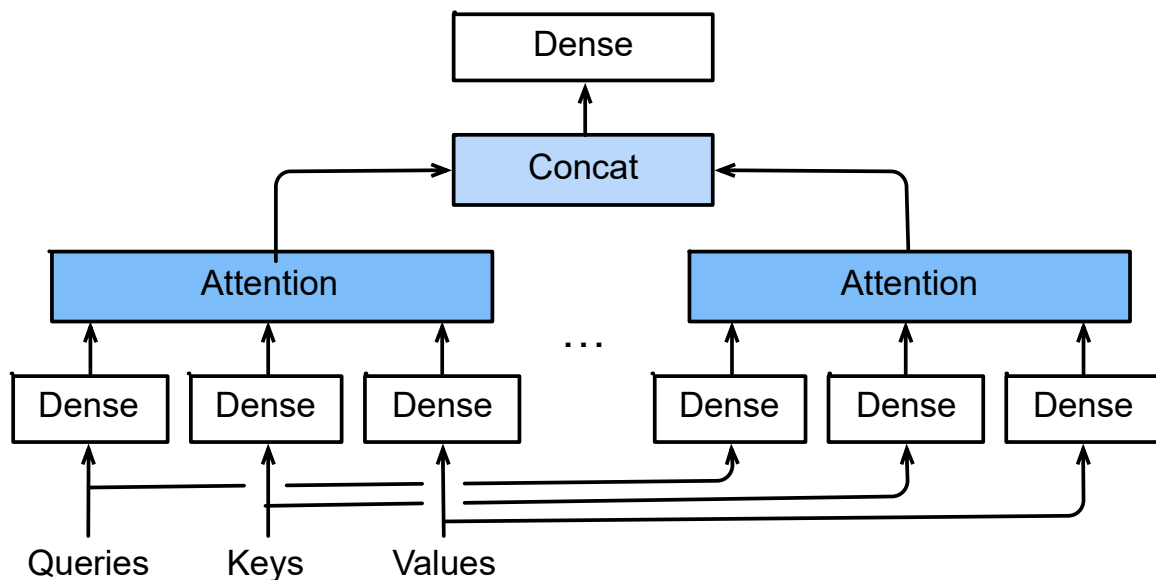
$$P_{i,2j} = \sin(i/10000^{2j/d})$$

$$P_{i,2j+1} = \cos(i/10000^{2j/d})$$

► 输出 $X + P$



多头注意力机制 (Multi-head Self-attention)

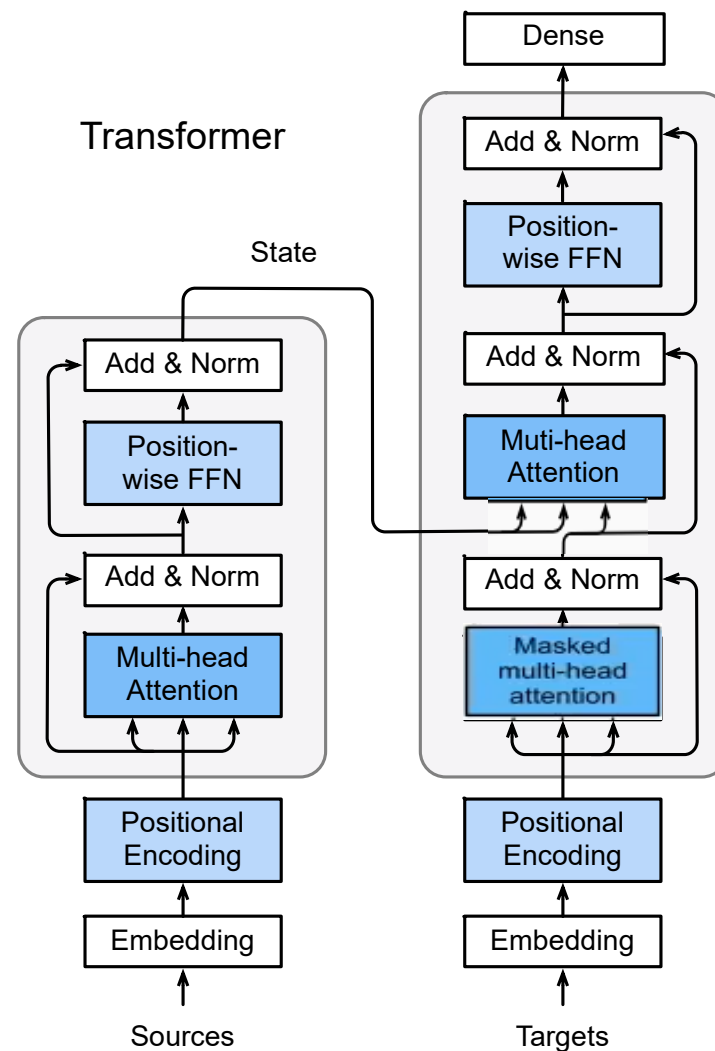


$$\mathbf{W}_q^{(i)} \in \mathbb{R}^{p_q \times d_q}, \mathbf{W}_k^{(i)} \in \mathbb{R}^{p_k \times d_k}, \text{ and } \mathbf{W}_v^{(i)} \in \mathbb{R}^{p_v \times d_v}$$

$$\mathbf{o}^{(i)} = \text{attention}(\mathbf{W}_q^{(i)} \mathbf{q}, \mathbf{W}_k^{(i)} \mathbf{k}, \mathbf{W}_v^{(i)} \mathbf{v})$$

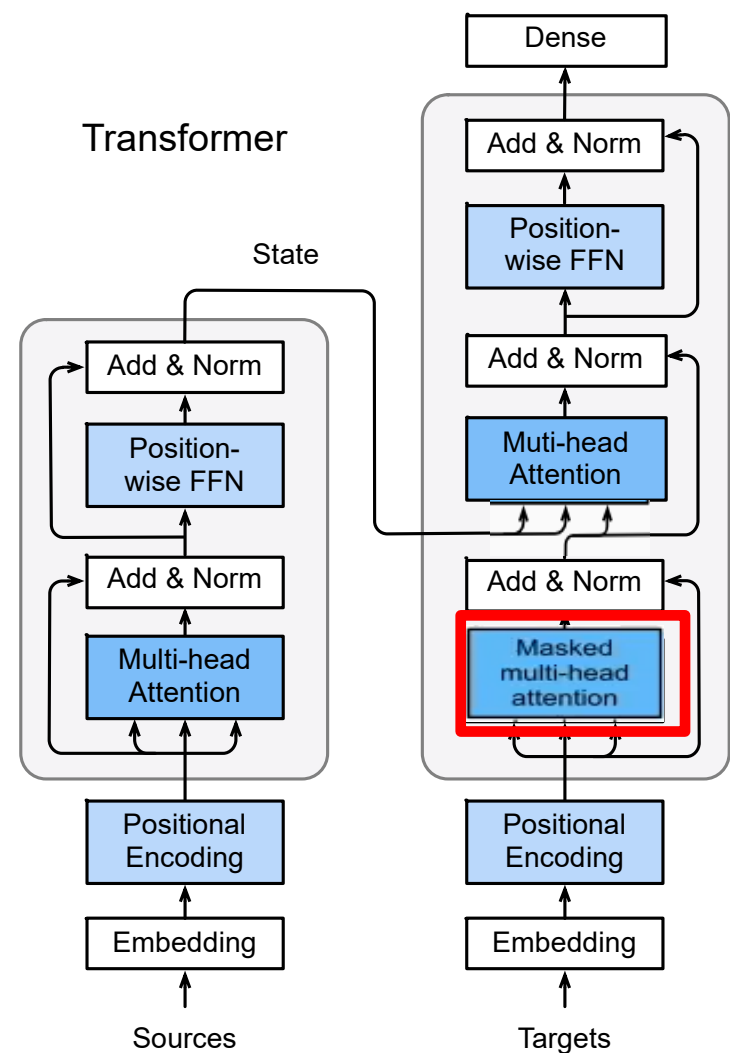
for $i = 1, \dots, h$

$$\mathbf{o} = \mathbf{W}_o \begin{bmatrix} \mathbf{o}^{(1)} \\ \vdots \\ \mathbf{o}^{(h)} \end{bmatrix}$$



多头注意力机制 (Multi-head Self-attention)

- ▶ 解码器对序列中一个元素输出时，不应该考虑该元素之后的元素
- ▶ 可以通过掩码来实现
 - ▶ 也就是计算 x_i 输出时，但假装当前序列长度为 i



位置前馈网络

► 将输入:

► 批量大小, 序列长度, 特征集大小

► 重新整形为:

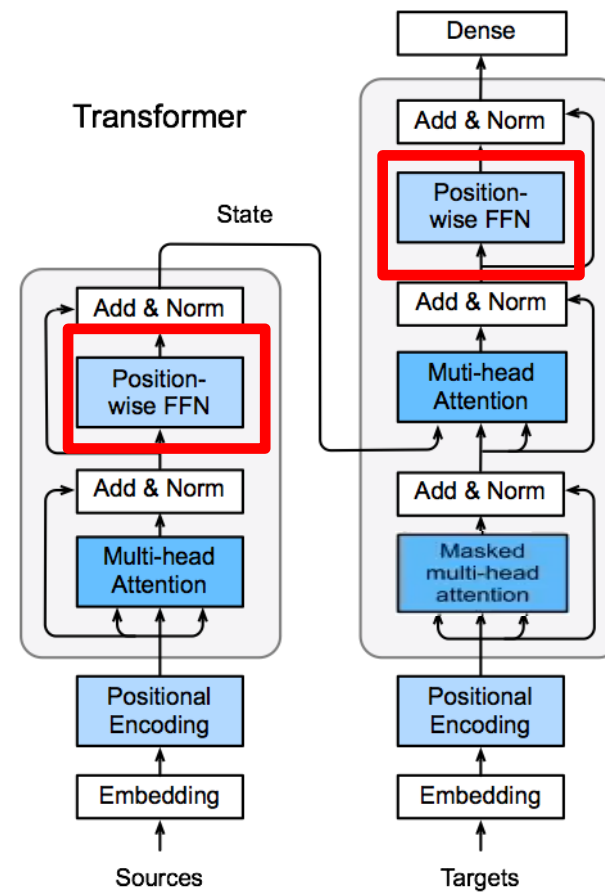
► 批量*序列长度, 特征集大小

► 将输入形状由 (b,n,d) 变换成 (bn,d)

► 作用两个全连接层

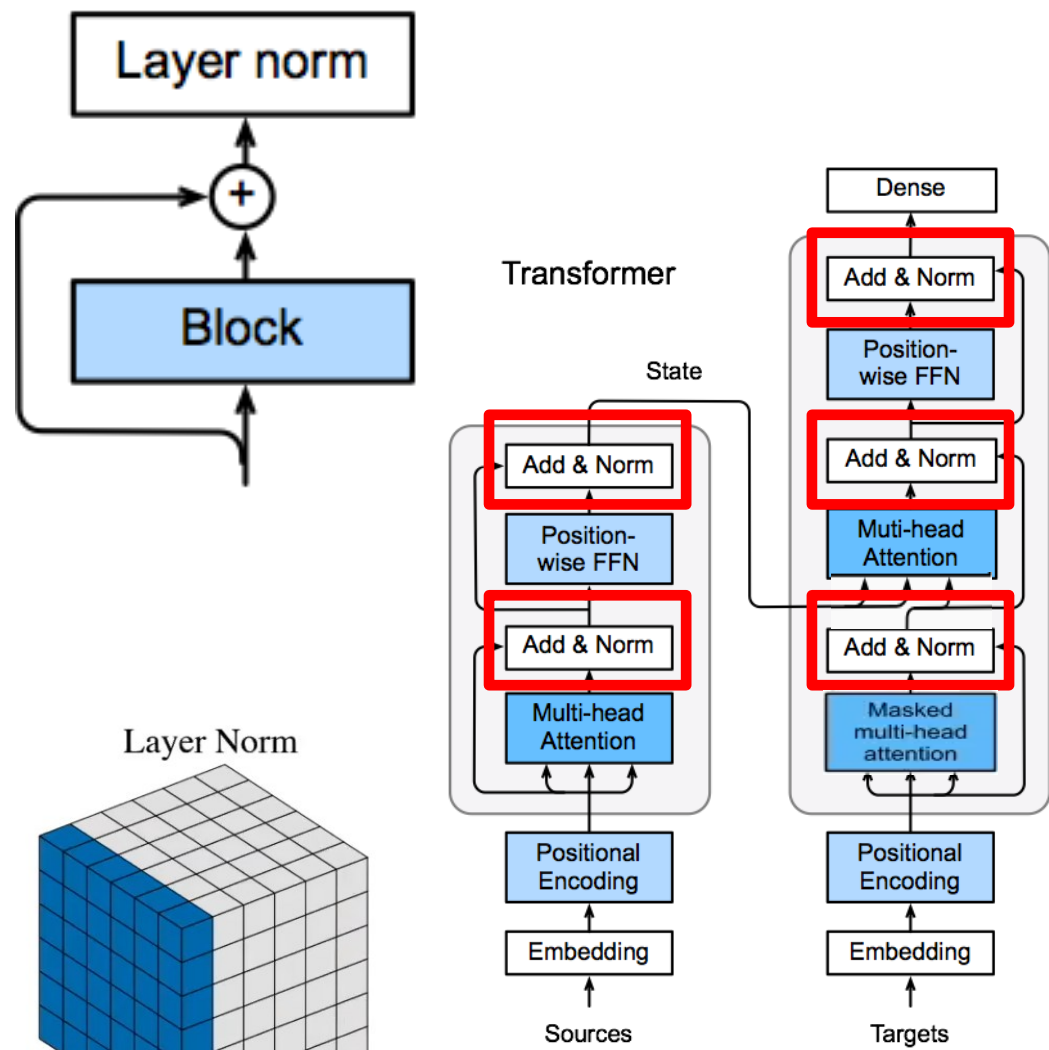
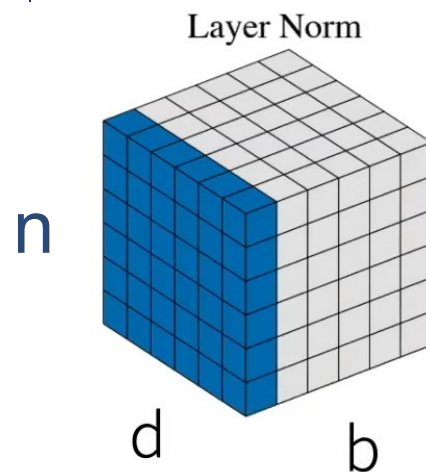
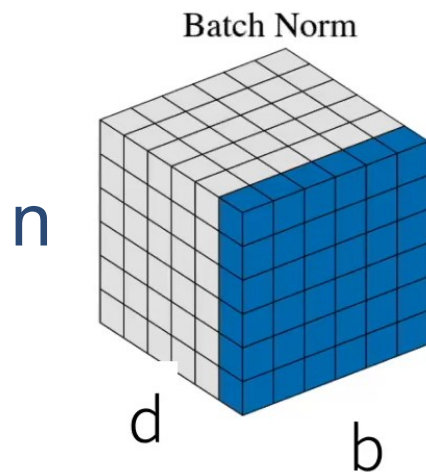
► 输出形状由 (bn,d) 变化回 (b,n,d)

► 等价于两层核窗口为1的一维卷积层



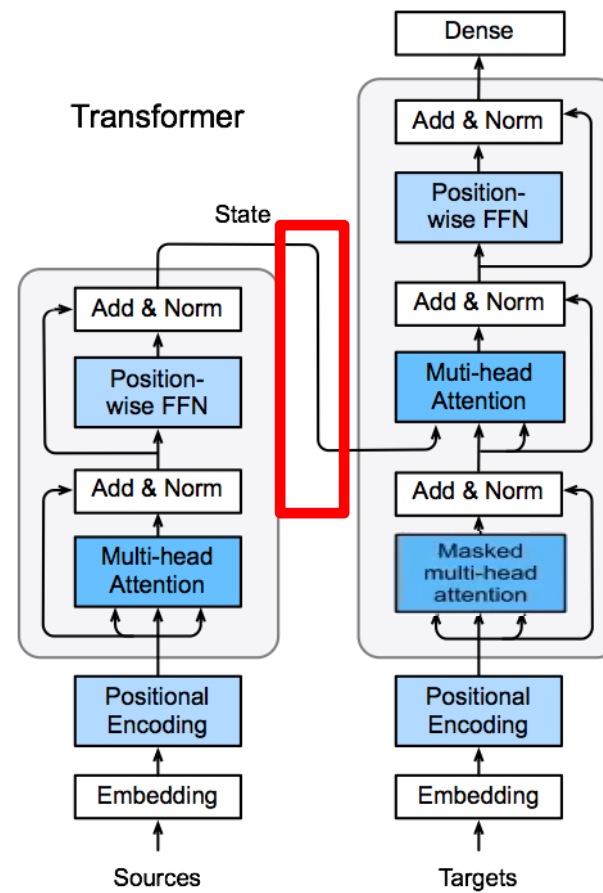
添加与归一化

- ▶ 批量归一化对每个特征/通道里元素进行归一化
- ▶ 不适合序列长度会变的NLP应用
- ▶ 层归一化对每个样本里的元素进行归一化



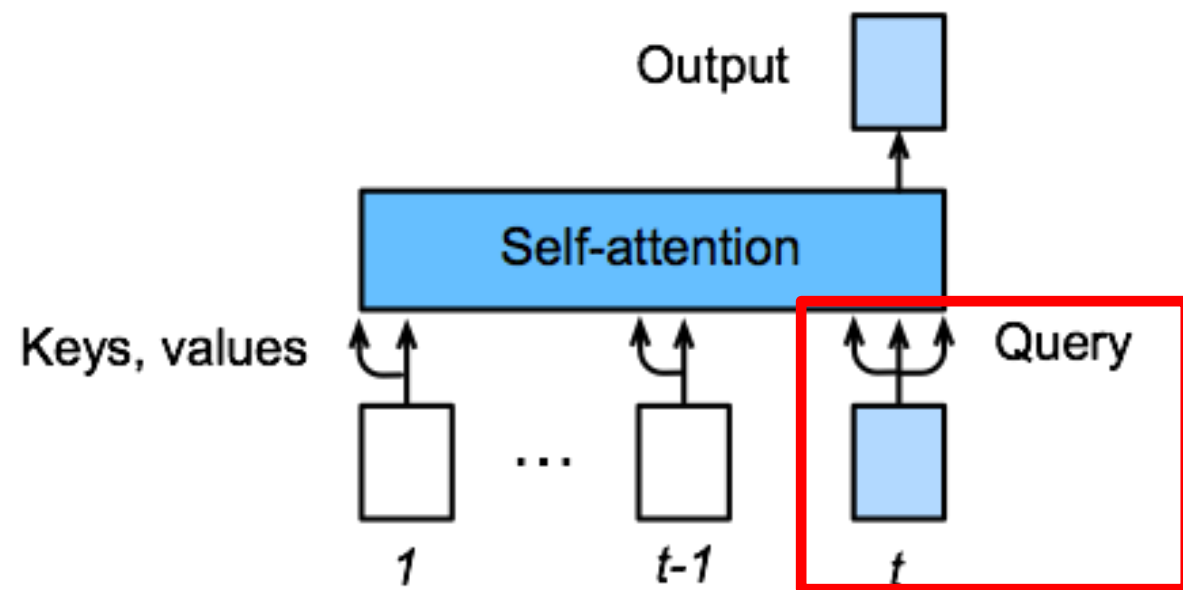
信息传递

- ▶ 编码器中的输出 $y_i, \dots, y_n$
- ▶ 将其作为解码中第 i 个Transformer块中多头注意力的key和value
- ▶ 它的query来自目标序列
- ▶ 意味着编码器和解码器中块的个数和输出维度都是一样的

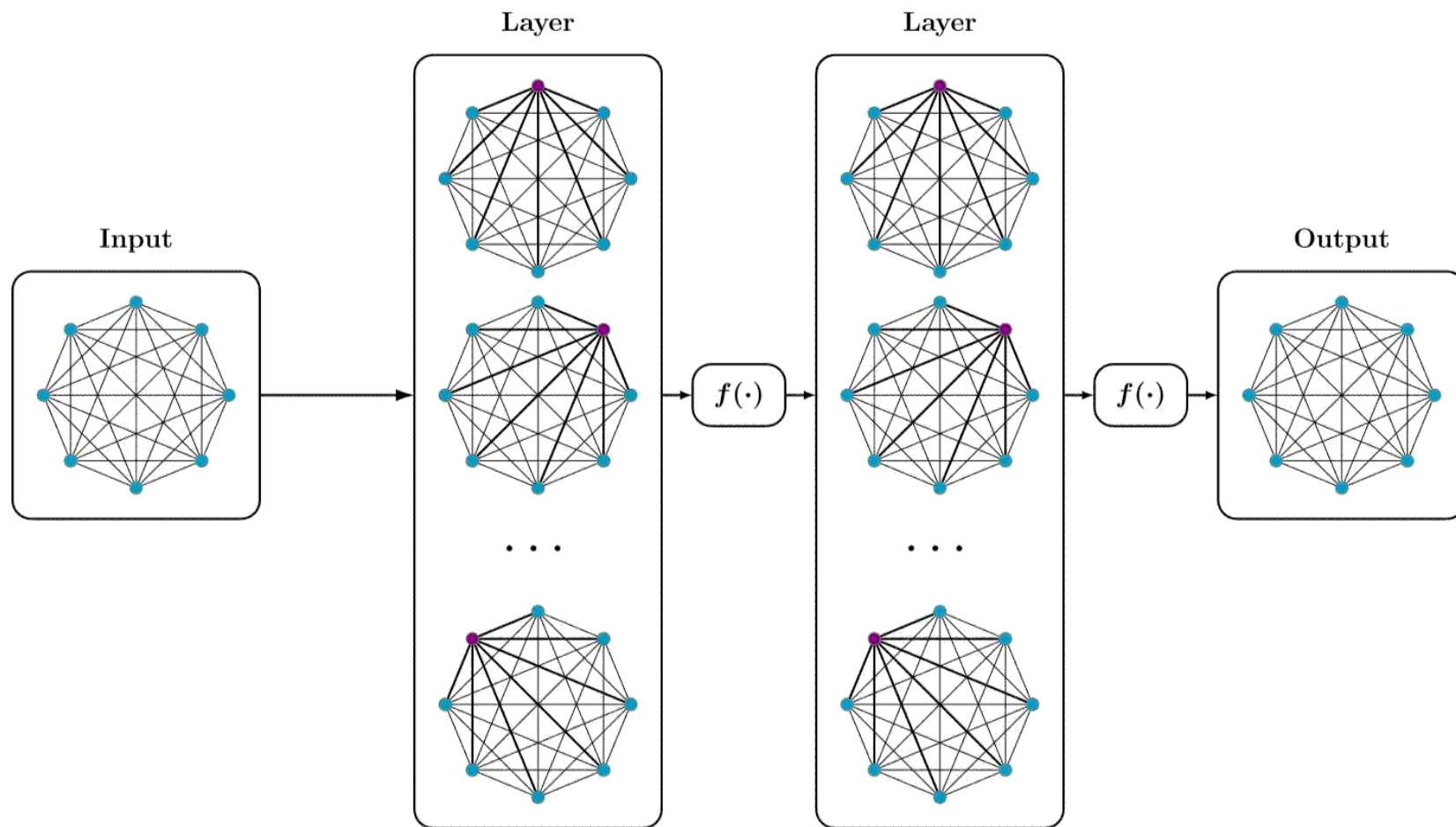


预测

- ▶ 预测第 $t+1$ 个输出时
- ▶ 解码器中输入前 t 个预测值
- ▶ 在自注意力中，前 t 个预测值作为key和value，第 t 个预测值还作为query



Transformer



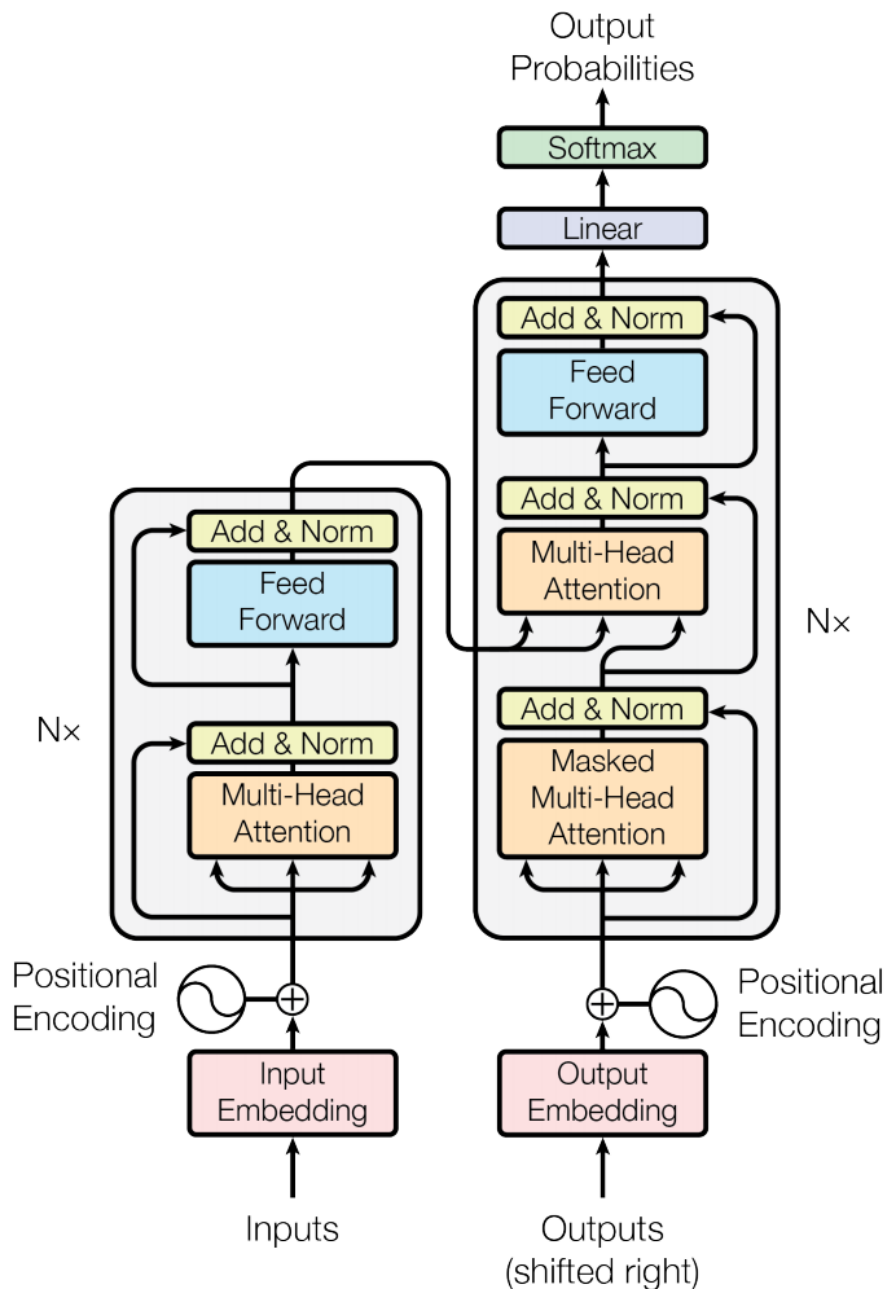
复杂度分析

模型	每层复杂度	序列操作数	最大路径长度
CNN	$O(knd^2)$	$O(1)$	$O(\log_k(n))$
RNN	$O(nd^2)$	$O(n)$	$O(n)$
Transformer	$O(n^2d)$	$O(1)$	$O(1)$

k 卷积核大小 n 序列长度 d 维度

总结

- ▶ Transformer是一个纯使用注意力的编码-解码器
- ▶ 编码器和解码器都有 n 个 transformer 块
- ▶ 每个块里使用多头(自)注意力
- ▶ 基于位置的前馈网络
- ▶ 层归一化





评价方法

BLEU

- ▶ BLEU (Bilingual Evaluation Understudy) 是衡量模型生成序列和参考序列之间的N元词组 (N-Gram) 的重合度。
- ▶ 最早用来评价机器翻译模型的质量，目前也广泛应用在各种序列生成任务中。
- ▶ P_N 是预测中所有 n-gram 的精度
- ▶ 标签序列 A B C D E F
- ▶ 预测序列 A B B C D
- ▶ 有 $p_1 = \frac{4}{5}, p_2 = \frac{3}{4}, p_3 = \frac{1}{3}, p_4 = 0$

BLEU

令 $\mathbf{x}$ 为从模型分布 p_θ 中生成的一个候选 (Candidate) 序列, $\mathbf{s}^{(1)}, \dots, \mathbf{s}^{(K)}$ 为从真实数据分布中采集的一组参考 (Reference) 序列, $\mathcal{W}$ 为从生成的候选序列中提取所有 **N 元组合** 的集合, 这些 N 元组合的精度 (Precision)

$$P_N(\mathbf{x}) = \frac{\sum_{w \in \mathcal{W}} \min(c_w(\mathbf{x}), \max_{k=1}^K c_w(\mathbf{s}^{(k)}))}{\sum_{w \in \mathcal{W}} c_w(\mathbf{x})}, \quad (15.39)$$

其中 $c_w(\mathbf{x})$ 是 N 元组合 w 在生成序列 $\mathbf{x}$ 中出现的次数, $c_w(\mathbf{s}^{(k)})$ 是 N 元组合 w 在参考序列 $\mathbf{s}^{(k)}$ 中出现的次数. N 元组合的精度 $P_N(\mathbf{x})$ 是计算生成序列中的 N 元组合有多少比例在参考序列中出现.

BLEU

BLEU 算法是通过计算不同长度的 N 元组合 ($N = 1, 2, \dots$) 的精度, 并进行几何加权平均而得到.

$$\text{BLEU-N}(\mathbf{x}) = b(\mathbf{x}) \times \exp\left(\sum_{N=1}^{N'} \alpha_N \log P_N\right), \quad (15.41)$$

长的匹配具有高权重

$$b(\mathbf{x}) = \begin{cases} 1 & \text{if } l_x > l_s \\ \exp(1 - l_s/l_x) & \text{if } l_x \leq l_s \end{cases}$$

惩罚过短的预测

其中 l_x 为生成序列 $\mathbf{x}$ 的长度, l_s 为参考序列的最短长度.

困惑度

- ▶ 困惑度 (Perplexity) 是信息论的一个概念，可以用来衡量一个分布的不确定性。
- ▶ 给定一个测试文本集合，一个好的序列生成模型应该使得测试集合中的句子的联合概率尽可能高。
- ▶ 困惑度可以衡量模型分布与样本经验分布之间的契合程度。困惑度越低则两个分布越接近。

困惑度

假设测试集合共有独立同分布的 N 个序列 $\{\mathbf{x}_{1:T_n}^{(n)}\}_{n=1}^N$ 。我们可以用模型 $p_\theta(\mathbf{x})$ 对每个序列计算其概率 $p_\theta(\mathbf{x}_{1:T_n}^{(n)})$ ，整个测试集的联合概率为

$$\prod_{n=1}^N p_\theta(\mathbf{x}_{1:T_n}^{(n)}) = \prod_{n=1}^N \prod_{t=1}^{T_n} p_\theta(x_t^{(n)} | \mathbf{x}_{1:(t-1)}^{(n)}). \quad (15.35)$$

模型 $p_\theta(\mathbf{x})$ 的困惑度定义为

$$\text{PPL}(\theta) = 2^{-\frac{1}{T} \sum_{n=1}^N \log p_\theta(\mathbf{x}_{1:T_n}^{(n)})} \quad (15.36)$$

$$= 2^{-\frac{1}{T} \sum_{n=1}^N \sum_{t=1}^{T_n} \log p_\theta(x_t^{(n)} | \mathbf{x}_{1:(t-1)}^{(n)})} \quad (15.37)$$

$$= \left(\prod_{n=1}^N \prod_{t=1}^{T_n} p_\theta(x_t^{(n)} | \mathbf{x}_{1:(t-1)}^{(n)}) \right)^{-1/T}, \quad (15.38)$$

其中 $T = \sum_{n=1}^N T_n$ 为测试数据集中序列的总长度。可以看出，困惑度为每个词条件概率 $p_\theta(x_t^{(n)} | \mathbf{x}_{1:(t-1)}^{(n)})$ 的几何平均数的倒数。测试集中所有序列的概率越大，困惑度越小，模型越好。

ROUGE

- ▶ ROUGE (Recall-Oriented Understudy for Gisting Evaluation)
最早应用于文本摘要领域。和 BLEU 类似，但 ROUGE 计算的是召回率 (Recall)。

假设从模型分布 p_θ 中生成一个候选 (Candidate) 序列 $\mathbf{x}$ ，从真实数据分布中采样出的一组参考 (Reference) 序列 $\mathbf{s}^{(1)}, \dots, \mathbf{s}^{(K)}$ ，令 $\mathcal{W}$ 为从参考序列中提取 N 元组合的集合，ROUGE- N 的定义为

$$\text{ROUGE-N}(\mathbf{x}) = \frac{\sum_{k=1}^K \sum_{w \in \mathcal{W}} \min(c_w(\mathbf{x}), c_w(\mathbf{s}^{(k)}))}{\sum_{k=1}^K \sum_{w \in \mathcal{W}} c_w(\mathbf{s}^{(k)})}, \quad (15.42)$$

其中 $c_w(\mathbf{x})$ 是 N 元组合 w 在生成序列 $\mathbf{x}$ 中出现的次数， $c_w(\mathbf{s}^{(k)})$ 是 N 元组合 w 在参考序列 $\mathbf{s}^{(k)}$ 中出现的次数。